



UNIVERSIDAD AUTÓNOMA DE QUERÉTARO
FACULTAD DE CONTADURÍA Y ADMINISTRACIÓN

PROYECTO FINAL

SQL HANDBOOK

ACTUARÍA

ARMANDO NÁJERA
ARREDONDO

309198

ÍNDICE

¿POR QUÉ APRENDER SQL?	6
1. CONCEPTOS BÁSICOS DE SQL	6
1.1. BASE DE DATOS RELACIONAL (RDBMS)	6
1.2. COMANDOS BÁSICOS DE SQL	6
2. CONSULTAS BÁSICAS CON SQL	6
2.1. SELECT: CONSULTAR DATOS	6
2.2. WHERE: FILTROS EN LAS CONSULTAS	7
2.3. ORDER BY: ORDENAR LOS RESULTADOS	7
3. MODIFICACIÓN DE DATOS CON SQL	7
3.1. INSERT: AGREGAR NUEVOS REGISTROS	7
3.2. UPDATE: MODIFICAR DATOS EXISTENTES	8
3.3. DELETE: ELIMINAR REGISTROS	8
4. RELACIONES ENTRE TABLAS (JOINS)	8
4.1. INNER JOIN	8
4.2. LEFT JOIN	9
5. FUNCIONES DE AGREGACIÓN Y AGRUPACIÓN	9
5.1. COUNT, SUM, AVG, MAX, MIN	9
6. SUBCONSULTAS	10
GUÍA DE INSTALACIÓN DE SQL SERVER MANAGEMENT STUDIO (SSMS)	10
PASO 1: REQUISITOS PREVIOS	10
REQUISITOS DE SISTEMA PARA SSMS:	10
PASO 2: DESCARGAR SQL SERVER MANAGEMENT STUDIO (SSMS)	11
PASO 3: EJECUTAR EL INSTALADOR DE SSMS	11
1. EJECUTA EL ARCHIVO DE INSTALACIÓN	11
2. CONFIGURACIÓN DE INSTALACIÓN	11
3. ESPERAR A QUE LA INSTALACIÓN FINALICE	11
4. FINALIZAR LA INSTALACIÓN	11
PASO 4: INICIAR SQL SERVER MANAGEMENT STUDIO (SSMS)	12
PASO 5: VERIFICAR LA INSTALACIÓN	12
PASO 6: ACTUALIZACIONES	13
TIPOS DE COMANDOS SQL: DDL, DML, DCL, Y TCL	13
1. DDL - DATA DEFINITION LANGUAGE (LENGUAJE DE DEFINICIÓN DE DATOS)	13
PRINCIPALES COMANDOS DDL	13
2. DML - DATA MANIPULATION LANGUAGE (LENGUAJE DE MANIPULACIÓN DE DATOS)	14
PRINCIPALES COMANDOS DML	14
3. DCL - DATA CONTROL LANGUAGE (LENGUAJE DE CONTROL DE DATOS)	15
PRINCIPALES COMANDOS DCL	15
4. TCL - TRANSACTION CONTROL LANGUAGE (LENGUAJE DE CONTROL DE TRANSACCIONES)	15
PRINCIPALES COMANDOS TCL	15
RESUMEN DE COMANDOS SQL	16
1. TIPOS DE DATOS NUMÉRICOS	17
A. INT (ENTERO)	17
B. DECIMAL O NUMERIC	17
C. FLOAT / DOUBLE	17

2. TIPOS DE DATOS DE TEXTO	18
A. VARCHAR.....	18
B. CHAR.....	18
C. TEXT.....	18
3. TIPOS DE DATOS DE FECHA Y HORA	19
A. DATE.....	19
B. DATETIME / TIMESTAMP	19
C. TIME	19
4. TIPOS DE DATOS BOOLEANOS.....	20
A. BOOLEAN / BOOL	20
5. TIPOS DE DATOS BINARIOS	20
A. BLOB	20
1. SINTAXIS BÁSICA DEL SELECT	21
2. SELECCIONAR TODAS LAS COLUMNAS (*)	21
3. FILTRAR LOS RESULTADOS CON WHERE.....	21
4. ORDENAR LOS RESULTADOS CON ORDER BY	22
5. LIMITAR LOS RESULTADOS CON LIMIT.....	22
6. CONSULTAS CON CONDICIONES MÚLTIPLES (AND, OR)	23
7. FUNCIONES DE AGREGACIÓN (SUM, AVG, COUNT, MAX, MIN)	23
8. AGRUPAR RESULTADOS CON GROUP BY.....	24
9. FILTRAR CON HAVING	24
1. USANDO WHERE PARA FILTRAR REGISTROS	24
INSTRUCCIONES	25
EJEMPLO	25
2. USANDO BETWEEN PARA FILTRAR POR RANGO DE VALORES	25
INSTRUCCIONES	25
EJEMPLO	25
3. USANDO IN PARA SELECCIONAR VARIAS OPCIONES.....	26
INSTRUCCIONES	26
EJEMPLO	26
4. USANDO IS NULL PARA FILTRAR VALORES NULOS	26
INSTRUCCIONES	26
EJEMPLO	26
5. USANDO LIKE PARA BÚSQUEDAS CON PATRONES (CADENAS DE TEXTO).	27
INSTRUCCIONES	27
EJEMPLOS.....	27
1. SINTAXIS BÁSICA DE ORDER BY	28
INSTRUCCIONES	28
SINTAXIS.....	28
2. ORDENAR POR UNA COLUMNA	29
INSTRUCCIONES	29
EJEMPLO	29
3. ORDENAR DE FORMA DESCENDENTE (DESC)	29
INSTRUCCIONES	29
EJEMPLO	29
4. ORDENAR POR VARIAS COLUMNAS.....	30
INSTRUCCIONES	30
EJEMPLO	30
5. ORDENAR FECHAS.....	30

INSTRUCCIONES	30
EJEMPLO	30
6. ORDENAR CADENAS DE TEXTO (ALFABÉTICAMENTE)	31
INSTRUCCIONES	31
EJEMPLO	31
SINTAXIS BÁSICA DE CREATE TABLE	31
SINTAXIS GENERAL	31
EJEMPLO 1: CREAR UNA TABLA SIMPLE	32
INSTRUCCIONES	32
SQL PARA CREAR LA TABLA	32
EJEMPLO 2: CREAR UNA TABLA CON UNA CLAVE PRIMARIA	32
INSTRUCCIONES	33
SQL PARA CREAR LA TABLA CON UNA CLAVE PRIMARIA	33
EJEMPLO 3: CREAR UNA TABLA CON CLAVES FORÁNEAS	33
INSTRUCCIONES	33
SQL PARA CREAR LAS TABLAS CON CLAVE FORÁNEA	33
EJEMPLO 4: CREAR UNA TABLA CON RESTRICCIONES DE UNICIDAD	34
INSTRUCCIONES	34
SQL PARA CREAR LA TABLA CON RESTRICCIÓN DE UNICIDAD	34
SINTAXIS BÁSICA DE UPDATE	35
SINTAXIS GENERAL	35
EJEMPLO 1: ACTUALIZAR UN SOLO REGISTRO	35
INSTRUCCIONES	35
SQL PARA ACTUALIZAR EL REGISTRO	35
EJEMPLO 2: ACTUALIZAR VARIOS REGISTROS	36
INSTRUCCIONES	36
SQL PARA ACTUALIZAR MÚLTIPLES REGISTROS	36
EJEMPLO 3: ACTUALIZAR VARIOS CAMPOS EN UN REGISTRO	36
INSTRUCCIONES	36
SQL PARA ACTUALIZAR MÚLTIPLES COLUMNAS	36
EJEMPLO 4: ACTUALIZAR TODOS LOS REGISTROS (SIN WHERE)	37
INSTRUCCIONES	37
SQL PARA ACTUALIZAR TODOS LOS REGISTROS	37
EJEMPLO 5: USAR SUBCONSULTAS PARA ACTUALIZAR DATOS	37
INSTRUCCIONES	37
SQL PARA ACTUALIZAR USANDO UNA SUBCONSULTA	37
CONSIDERACIONES IMPORTANTES AL USAR UPDATE	38
1. DROP: ELIMINAR UNA TABLA O UN OBJETO DE LA BASE DE DATOS	38
2. DELETE: ELIMINAR DATOS DE UNA TABLA CON CONDICIONES	39
3. TRUNCATE: ELIMINAR TODOS LOS DATOS DE UNA TABLA RÁPIDAMENTE	40
COMPARACIÓN ENTRE DROP, DELETE Y TRUNCATE	40
¿CUÁNDO USAR CADA UNO?	41
¿QUÉ PODEMOS HACER CON ALTER TABLE?	41
SINTAXIS BÁSICA DE ALTER TABLE	42
1. AGREGAR UNA COLUMNA A UNA TABLA	42
SINTAXIS	42
EJEMPLO DE USO	42
2. ELIMINAR UNA COLUMNA DE UNA TABLA	43
SINTAXIS	43

EJEMPLO DE USO	43
3. MODIFICAR EL TIPO DE DATOS DE UNA COLUMNA.....	43
SINTAXIS.....	43
EJEMPLO DE USO	43
4. RENOMBRAR UNA COLUMNA	44
SINTAXIS.....	44
EJEMPLO DE USO	44
5. RENOMBRAR LA TABLA.....	44
SINTAXIS.....	44
EJEMPLO DE USO	44
6. AGREGAR RESTRICCIONES (CONSTRAINTS)	45
SINTAXIS PARA AGREGAR UNA CLAVE PRIMARIA.....	45
EJEMPLO DE USO	45
SINTAXIS PARA AGREGAR UNA CLAVE FORÁNEA.....	45
EJEMPLO DE USO	45
RESUMEN DE OPERACIONES CON ALTER TABLE.....	45
OPERACIONES COMUNES CON VARIAS TABLAS EN SQL.....	46
1. USO DE JOIN PARA COMBINAR TABLAS	46
2. USO DE UNION PARA COMBINAR RESULTADOS DE CONSULTAS	48
3. SUBCONSULTAS (SUBQUERIES) PARA CONSULTAS ANIDADAS	48
4. OPERACIONES CON INNER JOIN, LEFT JOIN Y RIGHT JOIN	49
TIPOS DE UNIONES EN SQL.....	50
1. INNER JOIN.....	50
2. OUTER JOINS.....	51
3. CROSS JOIN	52
4. UNION VS UNION ALL	53
5. EXPRESIONES EN SQL.....	54
6. FUNCIONES PREDEFINIDAS EN BASES DE DATOS	55
AGRUPACIONES Y PARTICIONAMIENTO DE BASES DE DATOS EN SQL.....	55
AGRUPANDO DATOS (GROUP BY).....	55
FILTRANDO DATOS AGRUPADOS (HAVING)	56
CLÁUSULA OVER.....	57
FUNCIONES DE VENTANA (FIRST_VALUE, LAST_VALUE, LAG, LEAD)	57
ORDENAR REGISTROS AGRUPADOS (RANK, ROW_NUMBER).....	58
VISTAS (VIEWS)	59
FUNCIONES DE USUARIO (FUNCTIONS).....	59
PROCEDIMIENTOS ALMACENADOS (STORED PROCEDURES)	60
DISPARADORES (TRIGGERS)	61
ADMINISTRACIÓN DE BASES DE DATOS EN SQL.....	61
1. CÓMO CREAR UNA BASE DE DATOS	61
2. RESPALDAR UNA BASE DE DATOS	62
3. RESTAURAR UNA BASE DE DATOS	62
4. IMPORTAR Y EXPORTAR ARCHIVOS.....	63
5. ADMINISTRACIÓN DE PERMISOS.....	64
MODELADO DE UNA BASE DE DATOS RELACIONAL EN SQL.....	65
MODELO ENTIDAD-RELACIÓN (ER).....	65
LLAVES (PRIMARIAS Y SECUNDARIAS)	66
CONSTRAINTS (RESTRICCIONES)	67
ÍNDICES	69
NORMALIZACIÓN.....	69

SQL HANDBOOK

SQL (Structured Query Language) es el lenguaje estándar para interactuar con bases de datos. Nos permite realizar diversas operaciones sobre los datos almacenados en una base de datos relacional, como consultar, insertar, actualizar y eliminar datos.

¿Por qué aprender SQL?

- Es el lenguaje principal para interactuar con bases de datos.
- SQL es universal: se utiliza en MySQL, PostgreSQL, SQL Server, Oracle, entre otros.
- Permite gestionar grandes cantidades de datos de manera eficiente.

1. Conceptos Básicos de SQL

1.1. Base de Datos Relacional (RDBMS)

Una base de datos relacional organiza los datos en tablas, que están compuestas por filas (también llamadas registros) y columnas. Cada tabla tiene una clave primaria, que es un identificador único para cada fila.

Ejemplo de una tabla simple:

id_cliente	nombre	correo
1	Juan	juan@email.com
2	Maria	maria@email.com

1.2. Comandos Básicos de SQL

- **SELECT:** Recupera datos de una base de datos.
- **INSERT:** Inserta nuevos registros en una tabla.
- **UPDATE:** Modifica datos existentes.
- **DELETE:** Elimina registros.

2. Consultas Básicas con SQL

2.1. SELECT: Consultar Datos

La instrucción SELECT se utiliza para seleccionar datos de una base de datos. Puedes usarla para recuperar columnas específicas o todas las columnas de una tabla.

Sintaxis básica:

```
SELECT columna1, columna2 FROM  
tabla;
```

Ejemplo:

```
SELECT nombre, correo FROM clientes;
```

Este comando devolverá las columnas nombre y correo de la tabla clientes.

Imagen sugerida: Una tabla con datos ficticios y una flecha que señala los resultados de la consulta.

2.2. WHERE: Filtros en las Consultas

El comando WHERE se usa para filtrar los resultados según una condición. Sintaxis básica:

```
SELECT columna1, columna2 FROM tabla WHERE  
condición;
```

Ejemplo:

```
SELECT nombre, correo FROM clientes WHERE id_cliente =  
1;
```

Este comando devolverá los datos del cliente con id_cliente igual a 1.

Imagen sugerida: Un ejemplo de una tabla con un filtro aplicado (por ejemplo, resaltando una fila específica).

2.3. ORDER BY: Ordenar los Resultados

La cláusula ORDER BY se utiliza para ordenar los resultados de una consulta. Puedes ordenar los resultados de manera ascendente (ASC) o descendente (DESC).

Sintaxis básica:

```
SELECT columna1, columna2 FROM tabla ORDER BY columna1 ASC|  
DESC;
```

Ejemplo:

```
SELECT nombre, correo FROM clientes ORDER BY nombre ASC;
```

Este comando ordenará los resultados por el nombre de los clientes en orden alfabético ascendente.

3. Modificación de Datos con SQL

3.1. INSERT: Agregar Nuevos Registros

El comando INSERT INTO se utiliza para agregar nuevos registros a una tabla. Sintaxis básica:

```
INSERT INTO tabla (columna1, columna2) VALUES (valor1,  
valor2);
```

Ejemplo:

```
INSERT INTO clientes (id_cliente, nombre, correo) VALUES (3, 'Carlos',  
'carlos@email.com');
```

Este comando insertará un nuevo cliente con id_cliente igual a 3.

Imagen sugerida: Un diagrama que muestre cómo un nuevo registro es agregado a la tabla.

3.2. UPDATE: Modificar Datos Existentes

El comando UPDATE se utiliza para modificar registros existentes en una tabla. Sintaxis básica:

```
UPDATE tabla SET columna1 = valor1, columna2 = valor2 WHERE  
condición;
```

Ejemplo:

```
UPDATE clientes SET correo = 'nuevo_email@email.com' WHERE id_cliente = 1;
```

Este comando actualizará el correo del cliente con id_cliente igual a 1.

Imagen sugerida: Un ejemplo de una tabla con datos antes y después de un UPDATE.

3.3. DELETE: Eliminar Registros

El comando DELETE se usa para eliminar registros de

una tabla. Sintaxis básica:

```
DELETE FROM tabla WHERE
```

condición; Ejemplo:

```
DELETE FROM clientes WHERE id_cliente = 3;
```

Este comando eliminará el registro del cliente con id_cliente igual a 3.

Imagen sugerida: Una tabla donde una fila ha sido eliminada.

4. Relaciones entre Tablas (JOINS)

Las bases de datos relacionales suelen tener más de una tabla, y a menudo necesitamos combinar datos de estas tablas. Esto se hace mediante JOINS.

4.1. INNER JOIN

El INNER JOIN devuelve solo las filas que tienen coincidencias en ambas tablas. Sintaxis básica:

```
SELECT
```

```
columnas FROM
```

```
tabla1 INNER
```

```
JOIN tabla2
```

```
ON tabla1.columna =
```

```
tabla2.columna; Ejemplo:
```

```
SELECT clientes.nombre, pedidos.fecha
```

```
FROM clientes
```

```
INNER JOIN pedidos ON clientes.id_cliente = pedidos.id_cliente;
```

Este comando devuelve los nombres de los clientes y las fechas de sus pedidos.

Imagen sugerida: Un diagrama de dos tablas unidas por una relación, con los resultados visibles.

4.2. LEFT JOIN

El LEFT JOIN devuelve todas las filas de la primera tabla (izquierda) y las filas coincidentes de la segunda tabla (derecha). Si no hay coincidencia, los resultados de la segunda tabla serán NULL.

Sintaxis básica:

SELECT

columnas FROM

tabla1 LEFT

JOIN tabla2

ON tabla1.columna =

tabla2.columna; Ejemplo:

SELECT clientes.nombre, pedidos.fecha

FROM clientes

LEFT JOIN pedidos ON clientes.id_cliente = pedidos.id_cliente;

Este comando devolverá todos los clientes, incluso aquellos que no han realizado pedidos.

Imagen sugerida: Diagrama de tablas con el resultado de un LEFT JOIN, mostrando filas con valores NULL.

5. Funciones de Agregación y Agrupación

5.1. COUNT, SUM, AVG, MAX, MIN

Las funciones de agregación permiten realizar cálculos sobre los datos. Ejemplo:

SELECT COUNT(*) FROM clientes;

Esto devolverá el número total de clientes.

Ejemplo con agrupamiento:

SELECT ciudad, COUNT(*) FROM clientes GROUP BY

ciudad; Este comando contará cuántos clientes hay en cada ciudad.

Imagen sugerida: Un gráfico que muestre los resultados de una agregación (por ejemplo, el número de clientes por ciudad).

6. Subconsultas

Una subconsulta es una consulta dentro de otra consulta. Se utiliza para obtener resultados que se usan en la consulta principal.

Ejemplo:

```
<SELECT nombre FROM clientes WHERE id_cliente = (SELECT id_cliente  
FROM pedidos WHERE fecha = '2024-11-01');
```

Este comando devuelve el nombre del cliente que hizo un pedido en una fecha específica.

Imagen sugerida: Un diagrama que muestre cómo una subconsulta se inserta dentro de otra consulta más grande.

Guía de Instalación de SQL Server Management Studio (SSMS)

SQL Server Management Studio (SSMS) es una herramienta gráfica utilizada para administrar, configurar, desarrollar y administrar bases de datos SQL Server. Te permite interactuar con el servidor de bases de datos de una manera más sencilla, especialmente cuando trabajas en proyectos de desarrollo, mantenimiento y administración de bases de datos.

A continuación te guiaré paso a paso sobre cómo instalar **SQL Server Management Studio (SSMS)** en tu computadora, tanto en sistemas Windows como en macOS (aunque SSMS está diseñado principalmente para Windows).

Paso 1: Requisitos previos

Antes de instalar SSMS, asegúrate de cumplir con los requisitos mínimos:

Requisitos de sistema para SSMS:

- **Sistema operativo:** Windows 10 o posterior (en versiones anteriores de Windows, como Windows 7, también puede funcionar, pero es más recomendable tener versiones más recientes).
- **Procesador:** 1.4 GHz o superior.
- **Memoria RAM:** 2 GB como mínimo (se recomienda más, especialmente para trabajar con bases de datos grandes).
- **Espacio en disco:** Al menos 6 GB de espacio disponible en el disco duro.
- **Conexión a Internet:** Necesaria para descargar el instalador de SSMS y, si es necesario, las actualizaciones o componentes adicionales.

Paso 2: Descargar SQL Server Management Studio (SSMS) Para descargar **SSMS** de manera oficial, sigue estos pasos:

1. **Accede al sitio web oficial de Microsoft:**
 - o Dirígete al sitio web oficial de Microsoft para descargar SSMS:
Descargar SQL Server Management Studio
2. **Selecciona la versión más reciente:** En el sitio, verás un enlace para descargar la última versión de SSMS. Haz clic en el enlace que indica "Descargar" y se iniciará la descarga del archivo ejecutable .exe.
3. **Guardar el archivo:** El archivo descargado será un instalador llamado algo como SSMS-Setup-ENU.exe. Guárdalo en una ubicación fácil de recordar, como el escritorio o la carpeta de "Descargas".

Paso 3: Ejecutar el Instalador de SSMS

Una vez que se haya completado la descarga, es momento de iniciar el proceso de instalación.

1. Ejecuta el archivo de instalación

- Haz doble clic en el archivo SSMS-Setup-ENU.exe que descargaste.
- Si ves un mensaje de advertencia de seguridad (UAC - Control de

cuentas de usuario), haz clic en **Sí** para permitir que el instalador haga cambios en tu sistema.

2. Configuración de instalación

- El instalador te pedirá que confirmes la instalación de **SQL Server ManagementStudio**.
- La opción predeterminada de instalación suele ser la más adecuada. Si no necesitas personalizar la instalación, puedes dejar las opciones por defecto y hacer clic en **Instalar**.
- Si prefieres cambiar la ubicación de la instalación, puedes hacer clic en **Change...** y seleccionar una carpeta diferente en tu disco duro, pero generalmente la opción predeterminada está bien.

3. Esperar a que la instalación finalice

- La instalación de SSMS puede tardar unos minutos, dependiendo de la velocidad de tu computadora y conexión a Internet. Durante este proceso, el instalador descargará e instalará los componentes necesarios.
- Una vez que la instalación se complete, aparecerá un mensaje de confirmación.

4. Finalizar la instalación

- Cuando se complete la instalación, haz clic en **Cerrar** para finalizar el proceso.

Paso 4: Iniciar SQL Server Management Studio (SSMS)

Después de completar la instalación, puedes iniciar SSMS para comenzar a trabajar con tus bases de datos.

1. **Abrir SSMS:**

- Ve al menú **Inicio** de Windows.
- Busca **SQL Server Management Studio** o simplemente **SSMS**.
- Haz clic en el icono de **SQL Server Management Studio** para abrir la aplicación.

2. **Conectarse a una instancia de SQL Server:**

- Al abrir SSMS, se te pedirá que te conectes a un servidor de bases de datos.
- Si ya tienes SQL Server instalado en tu máquina local, puedes conectarte usando **localhost** o **127.0.0.1** como el nombre del servidor.
- Si estás trabajando con un servidor remoto, ingresa el nombre del servidor remoto o la dirección IP de la máquina.
- Asegúrate de que el tipo de autenticación esté configurado correctamente (por ejemplo, **Autenticación de SQL Server** o **Autenticación de Windows**).
- Una vez que hayas ingresado tus credenciales, haz clic en **Conectar**.

Paso 5: Verificar la instalación

Una vez conectado a SQL Server, puedes realizar algunas verificaciones básicas para asegurarte de que todo está funcionando correctamente:

1. **Crear una base de datos de prueba:**

- En la ventana de **Object Explorer** a la izquierda, haz clic derecho sobre **Databases** (Bases de datos) y selecciona **New Database**.
- En la ventana que aparece, dale un nombre a la base de datos (por ejemplo, TestDB) y haz clic en **OK**.

2. **Ejecutar una consulta básica:**

- Haz clic en **New Query** en la parte superior de la ventana.
- Escribe una consulta simple, como:

```
sql  
SELECT '¡Hola, SQL Server!' AS Mensaje;
```

- Haz clic en el botón **Ejecutar** o presiona **F5** para ejecutar la consulta. Deberías ver el resultado de la consulta en la ventana de resultados.

Paso 6: Actualizaciones

Es importante mantener SQL Server Management Studio actualizado para obtener las últimas características, correcciones de errores y mejoras de seguridad.

1. **Buscar actualizaciones manualmente:**

- Abre SSMS y ve a la barra de menú superior.
- Haz clic en **Tools** (Herramientas) y luego en **Check for Updates** (Buscar actualizaciones).
- Si hay una actualización disponible, se te pedirá que descargues e instales la nueva versión.

2. **Automatización:**

- También puedes habilitar la opción de actualizaciones automáticas para que SSMS se mantenga actualizado sin intervención manual.

Tipos de Comandos SQL: DDL, DML, DCL, y TCL

En SQL, los comandos se agrupan en categorías según su propósito y funcionalidad. Estas categorías son:

1. **DDL (Data Definition Language)** - Lenguaje de Definición de Datos
2. **DML (Data Manipulation Language)** - Lenguaje de Manipulación de Datos
3. **DCL (Data Control Language)** - Lenguaje de Control de Datos
4. **TCL (Transaction Control Language)** - Lenguaje de Control de Transacciones

1. DDL - Data Definition Language (Lenguaje de Definición de Datos)

Los **comandos DDL** se utilizan para definir y estructurar la base de datos. Esto incluye la creación, modificación y eliminación de objetos de la base de datos como **tablas, índices, vistas y esquemas**. Los cambios realizados con los comandos DDL son **permanentes** y afectan directamente la estructura de la base de datos.

Principales Comandos DDL:

1. **CREATE**: Crea una nueva tabla, base de datos, vista, índice, etc.

- **Ejemplo**: Crear una tabla.

```
CREATE TABLE empleados  
( id_empledo INT PRIMARY  
  KEY,nombre VARCHAR(100),  
  salario DECIMAL(10, 2));
```

2. **ALTER**: Modifica una estructura existente, como agregar, modificar o eliminar columnas de una tabla.

- **Ejemplo**: Agregar una columna a una tabla existente.

```
ALTER TABLE empleados ADD fecha_ingreso DATE;
```

3. **DROP**: Elimina un objeto de la base de datos, como una tabla, una vista o un índice.

- **Ejemplo**: Eliminar una tabla.

```
DROP TABLE empleados;
```

4. **TRUNCATE**: Elimina todos los registros de una tabla, pero conserva la estructura de la tabla para futuras inserciones.

- **Ejemplo**: Eliminar todos los registros de la tabla sin eliminar la tabla.

```
TRUNCATE TABLE empleados;
```


5. **RENAME**: Cambia el nombre de un objeto en la base de datos.

- **Ejemplo**: Renombrar una tabla.

```
RENAME TABLE empleados TO personal;
```

2. DML - Data Manipulation Language (Lenguaje de Manipulación de Datos)

Los **comandos DML** se utilizan para manipular los datos que están almacenados en las tablas. Esto incluye la inserción, actualización, eliminación y consulta de datos. Los cambios realizados con comandos DML son **temporales** y afectan solo los datos, no la estructura de la base de datos.

Principales Comandos DML:

1. **SELECT**: Recupera datos de una o más tablas.

- **Ejemplo**: Consultar todos los empleados.

```
SELECT id_empleado, nombre, salario FROM empleados;
```

2. **INSERT**: Inserta nuevos registros en una tabla.

- **Ejemplo**: Insertar un nuevo empleado.

```
INSERT INTO empleados (id_empleado, nombre,
salario) VALUES (1, 'Juan Pérez', 3000.00);
```

3. **UPDATE**: Modifica los datos existentes en una
tabla.

- **Ejemplo**: Actualizar el salario de un empleado.

```
UPDATE empleados SET salario = 3500.00 WHERE id_empleado = 1;
```

4. **DELETE**: Elimina registros existentes de una tabla.

- **Ejemplo**: Eliminar un empleado.

DELETE FROM empleados WHERE id_empleado = 1;

3. DCL - Data Control Language (Lenguaje de Control de Datos)

Los **comandos DCL** se utilizan para controlar el acceso a los datos y gestionar los permisos de los usuarios. Estos comandos permiten otorgar o revocar permisos a los usuarios y controlar quién tiene acceso a la base de datos.

Principales Comandos DCL:

1. **GRANT:** Otorga permisos a los usuarios sobre una base de datos o un objeto dentro de la base de datos.

- **Ejemplo:** Otorgar permisos de lectura y escritura a un usuario.

GRANT SELECT, INSERT, UPDATE ON empleados TO usuario1;

2. **REVOKE:** Revoca permisos previamente otorgados a los usuarios.

- **Ejemplo:** Revocar permisos de actualización de una tabla a un usuario.

REVOKE UPDATE ON empleados FROM usuario1;

3. **DENY:** Niega explícitamente ciertos permisos a los usuarios (es más restrictivo que

REVOKE).

- **Ejemplo:** Negar permisos de eliminación a un usuario.

DENY DELETE ON empleados TO usuario1;

4. TCL - Transaction Control Language (Lenguaje de Control de Transacciones)

Los **comandos TCL** se utilizan para gestionar las transacciones en la base de datos. Una transacción es un conjunto de operaciones SQL que se agrupan

juntas para ser ejecutadas de forma atómica (es decir, o todas se ejecutan o ninguna). Los comandos TCL permiten controlar el inicio, el compromiso y la reversión de las transacciones.

Principales Comandos TCL:

1. **COMMIT**: Confirma una transacción, haciendo que todos los cambios realizados durante la transacción sean permanentes.

- **Ejemplo**: Confirmar la transacción después de realizar un cambio.

```
COMMIT;
```

2. **ROLLBACK**: Revierte una transacción, deshaciendo todos los cambios realizados desde el último **COMMIT**.

- **Ejemplo**: Deshacer todos los cambios realizados en una transacción.

```
ROLLBACK;
```

3. **SAVEPOINT**: Establece un punto de control dentro de una transacción. Puedes hacer un **ROLLBACK** a este punto sin afectar el resto de la transacción.

- **Ejemplo**: Establecer un punto de guardado.

```
SAVEPOINT punto_guardado;
```

4. **SET TRANSACTION**: Establece las propiedades de la transacción, como el nivel de aislamiento (cómo se gestionan los bloqueos y el acceso concurrente).

- **Ejemplo**: Establecer el nivel de aislamiento de la transacción.

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Resumen de Comandos SQL

Tipo de Comando	Descripción	Comandos	Ejemplo
DDL	Definen la estructura de la base de datos (tablas, índices, etc.).	CREATE, ALTER, DROP, TRUNCATE, RENAME	CREATE TABLE empleados (...);
DML	Manipulan los datos almacenados en las tablas.	SELECT, INSERT, UPDATE, DELETE	INSERT INTO empleados VALUES (...);
DCL	Controlan los permisos y la seguridad de la base de datos.	GRANT, REVOKE, DENY	GRANT SELECT ON empleados TO usuario;
TCL	Gestionan las transacciones (agrupación de operaciones).	COMMIT, ROLLBACK, SAVEPOINT, SET TRANSACTION	COMMIT;

Tipos de datos:

Cuando trabajamos con bases de datos en SQL, es súper importante saber qué tipo de datos vamos a guardar en las tablas. Esto se hace a través de **tipos de datos**, que definen el tipo de valor que puede tomar una columna. Si no se define bien el tipo de dato, podemos tener problemas con la integridad de la base de datos, como guardar un texto donde se esperaba un número, o perder precisión en los cálculos.

Los tipos de datos en SQL pueden variar según el sistema de gestión de bases de datos (como MySQL, SQL Server, PostgreSQL, etc.), pero la mayoría de ellos son bastante similares. Aquí te voy a explicar los tipos de datos más comunes, categorizados para que los puedas entender mejor.

1. Tipos de Datos Numéricos

Los tipos de datos numéricos se usan para almacenar **números**. Dependiendo del tipo de número que necesites (si es un número entero o un número con decimales), vas a usar un tipo de dato u otro.

a. INT (Entero)

Este es el tipo de dato más común para almacenar números enteros, o sea, números sin decimales. Se usa, por ejemplo, para contar elementos, como el id de un usuario o el id de un producto.

- **Ejemplo:** Guardar el id de un usuario.

```
CREATE TABLE usuarios
(id INT PRIMARY KEY,
 nombre VARCHAR(100)
);
```

b. DECIMAL o NUMERIC

Si necesitas almacenar **números con decimales**, como precios o valores monetarios, deberías usar **DECIMAL** o **NUMERIC**. Ambos son muy similares, y definen cuántos dígitos totales y cuántos decimales va a tener el número.

- **Ejemplo:** Guardar un precio o salario.

```
CREATE TABLE productos
(id INT PRIMARY KEY,
 nombre VARCHAR(100),
 precio DECIMAL(10, 2) -- 10 dígitos en total, 2 después del punto decimal
);
```

c. FLOAT / DOUBLE

FLOAT y **DOUBLE** se usan para números con decimales, pero **FLOAT** tiene menos precisión que **DECIMAL**, mientras que **DOUBLE** tiene más precisión. Son más útiles cuando trabajas con **números muy grandes o muy pequeños**, como cálculos científicos o matemáticos.

- **Ejemplo:** Usar **FLOAT** para un valor con mucha precisión en cálculos científicos.

```
CREATE TABLE medidas
(id INT PRIMARY KEY,
 distancia FLOAT
);
```

2. Tipos de Datos de Texto

Los tipos de datos de texto se usan para almacenar **cadenas de caracteres**. Si vas a guardar cosas como nombres, direcciones, descripciones, etc., usarás estos tipos.

a. **VARCHAR**

VARCHAR es probablemente el tipo de dato de texto más usado. Se utiliza cuando el texto tiene un tamaño variable. En lugar de ocupar siempre el mismo espacio, solo ocupa el necesario para el texto que se guarda. Puedes especificar un número máximo de caracteres que puede tener la cadena.

- **Ejemplo:** Guardar el nombre de un usuario.

```
CREATE TABLE clientes
(id INT PRIMARY KEY,
 nombre VARCHAR(100) -- El nombre puede tener hasta 100
                        caracteres
);
```

b. CHAR

CHAR es otro tipo de texto, pero a diferencia de **VARCHAR**, el tamaño es fijo. Esto significa que siempre ocupará el mismo espacio, aunque el texto sea más corto.

Usualmente se usa cuando sabes que la longitud del texto será siempre la misma, como en códigos postales o códigos de país.

- **Ejemplo:** Guardar un código de país.

```
CREATE TABLE clientes
  (id INT PRIMARY KEY,
   codigo_pais CHAR(2) -- El código de país siempre tendrá 2
                        caracteres
  );
```

c. TEXT

El tipo **TEXT** se usa para guardar cadenas de texto muy largas, como descripciones, comentarios, artículos o cualquier otra cosa que no se pueda guardar en **VARCHAR**.

- **Ejemplo:** Guardar una descripción de un producto.

```
CREATE TABLE productos
  (id INT PRIMARY KEY,
   descripcion TEXT
  );
```

3. Tipos de Datos de Fecha y Hora

Cuando necesitas guardar fechas y horas (como la fecha de nacimiento de una persona, o el momento en que un pedido fue realizado), usarás los tipos de datos de **fecha y hora**.

a. DATE

DATE se usa para almacenar solo la **fecha** (año, mes y día) sin la hora.

- **Ejemplo:** Guardar la fecha de nacimiento de un usuario.

```
CREATE TABLE usuarios
  (id INT PRIMARY KEY,
   nombre
   VARCHAR(100),
   fecha_nacimiento
   DATE
  );
```

b. DATETIME / TIMESTAMP

DATETIME o **TIMESTAMP** se usan para almacenar tanto la **fecha** como la **hora** (año, mes, día, hora, minuto, segundo). La diferencia es que **TIMESTAMP** es más específico en algunas bases de datos, ya que puede almacenar también la zona horaria.

- **Ejemplo:** Guardar la fecha y hora en que se hizo un pedido.

```
CREATE TABLE pedidos
  (id INT PRIMARY KEY,
   id_cliente INT,
   fecha_pedido
   DATETIME
  );
```

c. TIME

TIME se usa solo para almacenar la **hora**, sin la fecha.

- **Ejemplo:** Guardar la hora de apertura de una tienda.

```
CREATE TABLE tienda (  
    id INT PRIMARY KEY,  
    hora_apertura TIME  
);
```

4. Tipos de Datos Booleanos

Los tipos **booleanos** son para almacenar valores de **verdadero o falso**. Algunos sistemas de bases de datos usan el tipo **BOOLEAN**, pero en muchos casos puedes usar **TINYINT**(con valores 0 o 1) para lograr lo mismo.

a. **BOOLEAN / BOOL**

Este tipo almacena valores lógicos: **TRUE** o **FALSE**.

- **Ejemplo:** Guardar si un producto está disponible o no.

```
CREATE TABLE productos  
(id INT PRIMARY KEY,  
    nombre  
    VARCHAR(100),  
    disponible BOOLEAN  
);
```

5. Tipos de Datos Binarios

Los tipos binarios se usan para almacenar **datos no textuales**, como imágenes, archivos o cualquier tipo de datos en formato binario.

a. **BLOB**

BLOB (Binary Large Object) es el tipo más común para almacenar grandes cantidades de datos binarios, como imágenes o videos.

- **Ejemplo:** Guardar una imagen de perfil de un usuario.

```
CREATE TABLE usuarios
(
  id INT PRIMARY KEY,
  nombre
  VARCHAR(100),
  foto_perfil BLOB
);
```

Tipos de datos más comunes que usarás en SQL:

- **Numéricos:** INT, DECIMAL, FLOAT, DOUBLE.
- **Texto:** VARCHAR, CHAR, TEXT.
- **Fecha y Hora:** DATE, DATETIME, TIME, TIMESTAMP.
- **Booleanos:** BOOLEAN, BOOL.
- **Binarios:** BLOB.

Consulta de Datos (SELECT)

Una de las operaciones más importantes cuando trabajamos con bases de datos es **consultar** o **recuperar** los datos que tenemos almacenados. Y para eso usamos el comando **SELECT**. Este comando te permite traer información de una o varias tablas, según lo que necesites. Es como cuando buscas algo en una hoja de Excel, pero en vez de hacer filtros manuales, lo haces con SQL.

1. Sintaxis Básica del SELECT

La sintaxis básica de una consulta SELECT es muy sencilla:

```
SELECT columna1,
columna2, ...FROM
nombre_de_tabla;
```

- **SELECT**: Especifica qué columnas quieres traer de la base de datos.
- **FROM**: Indica de qué tabla quieres obtener los datos.

Por ejemplo, si tenemos una tabla llamada clientes y queremos ver la columna nombre de todos los clientes, escribiríamos algo como esto:

```
SELECT  
nombre  
FROM  
clientes;
```

Esto nos va a devolver todos los nombres de la tabla clientes.

2. Seleccionar Todas las Columnas (*)

Si en lugar de especificar las columnas una por una, queremos obtener **todas las columnas** de una tabla, podemos usar el asterisco (*). Esto es muy útil cuando queremos ver toda la información que tenemos en una tabla.

Por ejemplo:

```
SELECT *  
FROM clientes;
```

Esto nos traerá todas las columnas de la tabla clientes (como id_cliente, nombre, direccion, etc.).

3. Filtrar los Resultados con WHERE

A veces no queremos ver todos los datos de una tabla, sino solo aquellos que cumplen ciertas condiciones. Para hacer esto, usamos la cláusula **WHERE**. Esto no permite filtrar los resultados según ciertos criterios.

Por ejemplo, si queremos ver solo los clientes que viven en una ciudad específica, digamos

"Madrid", podemos hacer lo siguiente:

```
SELECT nombre,  
ciudadFROM clientes  
WHERE ciudad = 'Madrid';
```

Esto nos va a devolver solo los clientes cuya columna ciudad sea igual a "Madrid".

4. Ordenar los Resultados con ORDER BY

Si necesitamos que los resultados de nuestra consulta estén ordenados, podemos usar **ORDER BY**. Podemos ordenar los datos en orden ascendente (de menor a mayor) o descendente (de mayor a menor) usando las palabras clave **ASC** (ascendente) o **DESC**(descendente).

Por ejemplo, si queremos ver los clientes ordenados alfabéticamente por su nombre, escribiríamos:

```
SELECT  
nombreFROM  
clientes  
ORDER BY nombre ASC;  
Si queremos ordenarlos de forma descendente (de la "Z" a la "A"),  
usaríamos:SELECT nombre  
FROM clientes  
ORDER BY nombre DESC;
```

5. Limitar los Resultados con LIMIT

A veces, no queremos ver todos los resultados de una tabla, sino solo una **parte** de ellos, por ejemplo, las primeras 10 filas. Para hacer esto, podemos usar **LIMIT** (en MySQL, PostgreSQL, etc.). Esto nos permite limitar la cantidad

de registros que devuelve tu consulta.

Ejemplo para obtener solo los primeros 5

clientes:SELECT nombre

FROM

clientesLIMIT

5;

6. Consultas con Condiciones Múltiples (AND, OR)

Si necesitamos hacer filtros más complejos, podemos usar los operadores **AND** y **OR** en tu cláusula WHERE para combinar varias condiciones.

- **AND**: Devuelve resultados solo si **todas** las condiciones se cumplen.
- **OR**: Devuelve resultados si **al menos una** de las condiciones se cumple.

Ejemplo usando **AND**:

SELECT nombre, ciudad,

edadFROM clientes

WHERE ciudad = 'Madrid' AND edad > 30;

Esto nos dará todos los clientes que vivan en Madrid **y** tengan más de 30 años.Ejemplo usando **OR**:

SELECT nombre, ciudad,

edadFROM clientes

WHERE ciudad = 'Madrid' OR ciudad = 'Barcelona';

Esto nos dará todos los clientes que vivan en **Madrid** o en **Barcelona**.

7. Funciones de Agregación (SUM, AVG, COUNT, MAX, MIN)

A veces, en lugar de mostrar todos los registros, necesitamos hacer **cálculos** u obtener información **resumida** de esos datos. Para eso, SQL tiene **funciones de agregación** comoCOUNT, SUM, AVG, MAX, MIN, entre otras.

- **COUNT**: Cuenta el número de registros.
- **SUM**: Suma los valores de una columna numérica.
- **AVG**: Calcula el promedio de una columna numérica.
- **MAX**: Devuelve el valor máximo.
- **MIN**: Devuelve el valor mínimo.

Ejemplo con **COUNT**:

```
SELECT COUNT(*)
FROM clientes;
```

Esto nos devolverá el número total de clientes en la tabla. Ejemplo con **SUM** (para sumar los precios de los productos):

sql

Copiar código

```
SELECT
SUM(precio)FROM
productos;
```

Esto nos devolverá la suma de los precios de todos los productos.

8. Agrupar Resultados con GROUP BY

Si queremos hacer cálculos agregados, pero agrupados por alguna categoría (por ejemplo, contar cuántos clientes hay en cada ciudad), podemos usar **GROUP BY**.

Ejemplo con **COUNT** y **GROUP BY** para contar el número de clientes por ciudad:

```
SELECT ciudad, COUNT(*)
FROM clientes
GROUP BY
ciudad;
```

Esto nos devolverá el número de clientes en cada ciudad.

9. Filtrar con HAVING

Cuando usamos **GROUP BY**, a veces necesitamos aplicar un filtro sobre los resultados agrupados. Para eso usamos **HAVING**, que es similar a **WHERE**, pero se aplica después de haber hecho el agrupamiento.

Ejemplo para mostrar solo las ciudades que tengan más de 5

clientes: `SELECT ciudad, COUNT(*)`

`FROM clientes`

`GROUP BY`

`ciudad`

`HAVING COUNT(*) > 5;`

Consulta de datos con condiciones (WHERE, BETWEEN, IN, IS NULL, LIKE)

Cuando trabajemos con bases de datos, uno de los aspectos más importantes será aprender a **filtrar** los datos que necesitamos con precisión. SQL nos ofrece varias maneras de hacerlo usando condiciones específicas. A continuación, les voy a explicar cómo podemos utilizar las cláusulas **WHERE**, **BETWEEN**, **IN**, **IS NULL**, y **LIKE** para hacer consultas más específicas y obtener exactamente los datos que necesitamos.

1. Usando WHERE para Filtrar Registros

La cláusula **WHERE** es la forma básica de filtrar los registros en una consulta SQL. Con ella, podemos especificar una condición que debe cumplirse para que los datos sean devueltos.

Instrucciones:

- Cuando queramos filtrar registros, utilizaremos **WHERE** para especificar una condición que debe cumplirse.
- La condición puede ser cualquier comparación (como `=`, `>`, `<`, `<>`, etc.) o incluso una combinación de varias condiciones usando **AND** o **OR**.

Ejemplo:

Si necesitamos ver todos los empleados que trabajan en el departamento de ventas, podemos escribir:

```
SELECT nombre,  
departamento  
FROM  
empleados  
WHERE departamento = 'Ventas';
```

Esto nos devolverá solo los registros donde la columna **departamento** tenga el valor 'Ventas'.

2. Usando BETWEEN para Filtrar por Rango de Valores

Si necesitamos filtrar los datos dentro de un **rango de valores** (por ejemplo, fechas o números entre un valor mínimo y máximo), podemos usar la cláusula **BETWEEN**. Este operador nos permite seleccionar registros cuyo valor esté dentro de un rango específico, incluyendo los valores de los extremos.

Instrucciones:

- Cuando necesitemos seleccionar registros dentro de un rango, usaremos **BETWEEN**.
- Recuerden que este operador **incluye** los valores límites, tanto el valor mínimo como el máximo.

Ejemplo:

Si necesitamos obtener todos los productos cuyo precio esté entre 10 y 50, podemos usar:

```
SELECT nombre, precio  
FROM productos  
WHERE precio BETWEEN 10 AND 50;
```

Esto nos devolverá todos los productos cuyo precio esté en ese rango (incluyendo 10 y 50).

3. Usando IN para Seleccionar Varias Opciones

La cláusula **IN** es muy útil cuando necesitamos comprobar si un valor está dentro de una lista de valores posibles. Es una forma más compacta y eficiente de escribir varias condiciones con **OR**.

Instrucciones:

- Cuando tengamos varios valores posibles y queramos seleccionar registros que coincidan con al menos uno de esos valores, usaremos **IN**.
- **IN** puede usarse con números, cadenas de texto, fechas, y más.

Ejemplo:

Si necesitamos ver todos los empleados que trabajan en los departamentos de "Ventas", "Marketing" o "Desarrollo", podemos escribir:

```
SELECT nombre,  
departamento  
FROM  
empleados  
WHERE departamento IN ('Ventas', 'Marketing', 'Desarrollo');
```

Esto nos devolverá todos los empleados que trabajen en cualquiera de esos tres departamentos.

4. Usando IS NULL para Filtrar Valores Nulos

En algunas situaciones, los datos en nuestras tablas pueden ser **NULL**. Esto indica que no hay valor asignado a esa columna. Si necesitamos filtrar los registros con valores **NULL** en una columna específica, debemos usar la cláusula **IS NULL**.

Instrucciones:

- Cuando queramos encontrar registros con valores **NULL** en una columna, utilizaremos **IS NULL**.
- Si necesitamos buscar registros donde una columna no sea **NULL**, usaremos **IS NOT NULL**.

Ejemplo:

Si necesitamos ver todos los productos que **no** tienen fecha de vencimiento registrada (es decir, donde la columna fecha_vencimiento es NULL), podemos escribir:

```
SELECT nombre,  
fecha_vencimiento FROM  
productos  
WHERE fecha_vencimiento IS NULL;
```

Esto nos devolverá todos los productos donde la columna **fecha_vencimiento** no tiene ningún valor asignado.

5. Usando LIKE para Búsquedas con Patrones (Cadenas de Texto)

Cuando necesitemos hacer búsquedas más flexibles con **cadenas de texto**, podemos usar el operador **LIKE**. Este operador nos permite buscar patrones dentro de los datos. Usando los **comodines** % (para representar cualquier cadena de caracteres) y _ (para un solo carácter), podremos realizar búsquedas parciales o con coincidencias específicas.

Instrucciones:

- Usaremos **LIKE** para buscar coincidencias en cadenas de texto.
- El comodín % reemplaza cualquier número de caracteres, incluso cero caracteres.
- El comodín _ reemplaza un solo carácter.

Ejemplos:

1. Si queremos encontrar todos los productos cuyo nombre empiece con "Cam", podemos escribir:

```
SELECT  
nombreFROM  
productos  
WHERE nombre LIKE 'Cam%';
```

Esto nos devolverá todos los productos cuyo nombre empiece con "Cam", como "Cámara", "Camisa", "Camiseta", etc.

2. Si queremos encontrar todos los productos cuyo nombre contenga "luz", podemos escribir:

```
SELECT  
nombreFROM  
productos  
WHERE nombre LIKE '%luz%';
```

Esto nos devolverá todos los productos cuyo nombre contenga la palabra "luz", como "Luz LED", "Luz cálida", etc.

3. Si buscamos productos cuyo nombre tenga exactamente tres letras, podemos usar el comodín _:

```
SELECT  
nombreFROM  
productos  
WHERE nombre LIKE '___';
```

Esto nos devolverá todos los productos cuyo nombre tenga exactamente tres caracteres.

Cuando queramos filtrar los resultados de nuestras consultas SQL de manera efectiva, podemos usar las siguientes cláusulas:

- **WHERE:** Para filtrar según una condición específica.
- **BETWEEN:** Para filtrar por un rango de valores.
- **IN:** Para seleccionar registros que coincidan con varios valores posibles.
- **IS NULL:** Para encontrar registros con valores **NULL**.
- **LIKE:** Para buscar coincidencias de patrones en cadenas de texto.

Ordenar registros (ORDER BY)

Cuando trabajemos con bases de datos, a menudo necesitaremos ver los resultados de nuestras consultas de manera ordenada. La cláusula **ORDER BY** en SQL es lo que utilizamos para ordenar los registros según uno o más criterios. Esto nos permite obtener la información de una manera más clara y útil, ya sea alfabéticamente, numéricamente o por fecha.

A continuación, se explicará cómo usar **ORDER BY** para organizar los resultados de nuestras consultas de la manera que más nos convenga.

1. Sintaxis Básica de ORDER BY

La sintaxis básica para usar **ORDER BY** es bastante simple. Básicamente, indicamos el nombre de la columna por la cual queremos ordenar y si lo haremos de forma ascendente (de menor a mayor) o descendente (de mayor a menor).

Instrucciones:

- Usaremos **ORDER BY** seguido del nombre de la columna o columnas que queremos utilizar para ordenar los registros.
- Por defecto, **ORDER BY** ordena los resultados en orden **ascendente** (de

menor a mayor), pero podemos especificar **DESC** si queremos un orden **descendente** (de mayor a menor).

- Si queremos ordenar por varias columnas, simplemente las separamos por comas y SQL ordenará los registros primero por la primera columna, luego por la segunda, y así sucesivamente.

Sintaxis:

```
SELECT columna1,  
columna2, ...FROM  
nombre_de_tabla  
ORDER BY columna1 [ASC|DESC], columna2 [ASC|DESC], ...;
```

2. Ordenar por una Columna

La forma más común de ordenar los registros es por una sola columna.

Supongamos que tenemos una tabla llamada **empleados** y queremos ordenar los resultados por el nombre de los empleados en orden alfabético.

Instrucciones:

- Para ordenar de manera ascendente, simplemente escribimos **ORDER BY** seguido del nombre de la columna.
- Si no especificamos **ASC**, el orden será ascendente por defecto.

Ejemplo:

Si queremos ver todos los empleados ordenados alfabéticamente por su **nombre**, escribiríamos:

```
SELECT nombre,  
departamentoFROM  
empleados  
ORDER BY nombre;
```

Esto devolverá todos los registros de la tabla **empleados** ordenados de la **A a la Z** por el nombre de los empleados.

3. Ordenar de Forma Descendente (DESC)

En ocasiones, necesitaremos ordenar los registros en orden descendente (de mayor a menor, o de la Z a la A). Para esto, usamos la palabra clave **DESC** (de "descending" en inglés).

Instrucciones:

- Para ordenar de mayor a menor o de la Z a la A, escribimos **ORDER BY** seguido de la columna que queremos ordenar y luego **DESC**.

Ejemplo:

Si queremos ordenar los empleados por **salario** de mayor a menor, escribiríamos: `SELECT nombre, salario`
`FROM empleados`
`ORDER BY salario DESC;`

Esto nos devolverá todos los empleados ordenados desde el salario más alto hasta el más bajo.

4. Ordenar por Varias Columnas

A veces, no solo necesitamos ordenar los registros por una columna, sino por varias. Por ejemplo, podríamos querer ordenar los empleados primero por **departamento** (de forma alfabética) y luego por **salario** (de mayor a menor) dentro de cada departamento.

Instrucciones:

- Para ordenar por varias columnas, simplemente agregamos más columnas después de **ORDER BY**.

- SQL ordenará primero por la primera columna, luego por la segunda, y así sucesivamente. Recuerden que cada columna puede tener su propio criterio de orden(ascendente o descendente).

Ejemplo:

Si queremos ordenar los empleados por **departamento** (de la A a la Z) y luego por **salario**

(de mayor a menor) dentro de cada departamento, escribimos:

```
SELECT nombre, departamento,  
salarioFROM empleados  
ORDER BY departamento ASC, salario DESC;
```

Esto nos devolverá los empleados primero organizados por **departamento** alfabéticamentey, dentro de cada departamento, ordenados por **salario** de mayor a menor.

5. Ordenar Fechas

Cuando trabajemos con fechas, también podemos ordenar los registros según la fecha. Esto es útil, por ejemplo, cuando necesitamos ver los registros más recientes o más antiguos.

Instrucciones:

- Si tenemos una columna de tipo **fecha**, podemos usar **ORDER BY** para ordenar los registros por esa columna, ya sea en orden ascendente o descendente.
- Si queremos ver las fechas más recientes primero, ordenamos de forma **descendente**.

Ejemplo:

Si tenemos una tabla **pedidos** con una columna de **fecha_pedido**, y queremos ver los pedidos más recientes, escribiríamos:

```
SELECT id_pedido,  
fecha_pedido  
FROM pedidos  
ORDER BY fecha_pedido DESC;
```

Esto nos devolverá los pedidos organizados desde el más reciente hasta el más antiguo.

6. Ordenar Cadenas de Texto (Alfabéticamente)

Las columnas que contienen **cadenas de texto** (como nombres, direcciones, ciudades, etc.) también pueden ser ordenadas alfabéticamente usando **ORDER BY**. Recuerden que por defecto, SQL ordena las cadenas de texto de forma ascendente, de la **A a la Z**.

Instrucciones:

- Cuando necesitemos ordenar cadenas de texto (como nombres, direcciones o categorías), simplemente usamos **ORDER BY** seguido de la columna correspondiente.

Ejemplo:

Si queremos ordenar los productos alfabéticamente por su nombre, escribimos:

```
SELECT nombre_producto  
FROM productos  
ORDER BY nombre_producto;
```

Esto nos devolverá los productos ordenados de la **A**

a la Z. Creación de tablas (CREATE)

Cuando trabajemos con bases de datos, uno de los primeros pasos que tomaremos será crear las **tablas** donde almacenaremos nuestros datos. En SQL,

usamos la sentencia **CREATE TABLE** para definir una nueva tabla y especificar las columnas que tendrá, junto con el tipo de datos que contendrán. Esta es una de las acciones más fundamentales cuando estamos diseñando una base de datos, porque las tablas actúan como los contenedores donde almacenamos toda la información.

Sintaxis Básica de CREATE TABLE

La sintaxis básica para crear una tabla en SQL es bastante simple, pero hay varios aspectos a tener en cuenta, como el nombre de la tabla, los nombres de las columnas, los tipos de datos, y las restricciones que podrían aplicarse a cada columna.

Sintaxis General:

```
CREATE TABLE nombre_de_tabla  
  ( nombre_columna1 tipo_de_dato  
    restricción,nombre_columna2  
    tipo_de_dato restricción,  
    ...  
  );
```

- **nombre_de_tabla**: El nombre de la tabla que estamos creando.
 - **nombre_columnaX**: Los nombres de las columnas que tendrá la tabla.
 - **tipo_de_dato**: El tipo de dato que contendrá cada columna (como INT, VARCHAR, DATE, etc.).
 - **restricción**: Opcionalmente, pueden añadirse restricciones a las columnas, como **NOT NULL**, **PRIMARY KEY**, **UNIQUE**, **FOREIGN KEY**, etc.

Ejemplo 1: Crear una Tabla Simple

Supongamos que queremos crear una tabla llamada **empleados** que almacene la información de los empleados de una empresa. Esta tabla

tendrá tres columnas: **id_empleado**, **nombre** y **departamento**.

Instrucciones:

- Vamos a definir la columna **id_empleado** como un número entero (**INT**) que será único para cada empleado.
- La columna **nombre** será de tipo **VARCHAR**, lo que nos permite almacenar cadenas de texto, y tendrá un tamaño máximo de 100 caracteres.
- La columna **departamento** también será de tipo **VARCHAR**, con un tamaño máximo de 50 caracteres.

SQL para Crear la Tabla:

```
CREATE TABLE empleados  
( id_empleado INT NOT  
  NULL,nombre  
  VARCHAR(100),  
  departamento  
  VARCHAR(50)  
);
```

En este caso:

- **id_empleado INT NOT NULL** significa que la columna **id_empleado** almacenará números enteros y no puede tener valores nulos.
- **nombre VARCHAR(100)** significa que la columna **nombre** almacenará texto (hasta 100 caracteres).
- **departamento VARCHAR(50)** significa que la columna **departamento** almacenará texto (hasta 50 caracteres).

Ejemplo 2: Crear una Tabla con una Clave Primaria

Ahora, supongamos que queremos asegurarnos de que cada **id_empleado** sea único para cada empleado y no se repita. Para esto, podemos definir **id_empleado** como una **PRIMARY KEY**. Esto también garantizará que no se puedan insertar registros con el mismo valor en esa columna.

Instrucciones:

- **PRIMARY KEY** asegura que los valores de esa columna sean únicos y no nulos.
- Usaremos **NOT NULL** para indicar que la columna **nombre** no puede tener valores nulos.

SQL para Crear la Tabla con una Clave Primaria:

```
CREATE TABLE empleados (  
    id_empleado INT NOT NULL PRIMARY  
    KEY,nombre VARCHAR(100) NOT NULL,  
    departamento VARCHAR(50)  
);
```

En este caso:

- **id_empleado INT NOT NULL PRIMARY KEY** significa que la columna **id_empleado** es única para cada registro y no puede ser nula.
- **nombre VARCHAR(100) NOT NULL** asegura que no podemos insertar un registro sin un valor en la columna **nombre**.
- **departamento VARCHAR(50)** sigue siendo una columna opcional (puede ser nula).

Ejemplo 3: Crear una Tabla con Claves Foráneas

Si estamos trabajando con varias tablas relacionadas, es posible que queramos establecer una relación entre ellas. Para esto, podemos usar una **clave foránea (FOREIGN KEY)**, que es una columna en una tabla que se refiere a la clave primaria de otra tabla.

Instrucciones:

- Suponiendo que tenemos una tabla llamada **departamentos**, y queremos asociar cada empleado con un departamento en particular.
- La columna **id_departamento** en la tabla **empleados** será una clave

foránea que hará referencia a la columna **id_departamento** en la tabla **departamentos**.

SQL para Crear las Tablas con Clave Foránea:

Primero creamos la tabla

```
departamentos:CREATE TABLE  
departamentos (  
    id_departamento INT NOT NULL PRIMARY  
    KEY, nombre_departamento VARCHAR(100)  
    NOT NULL  
);
```

Luego creamos la tabla **empleados** con una **clave foránea** que hace referencia a la tabla

departamentos:

```
CREATE TABLE empleados (  
    id_employado INT NOT NULL PRIMARY  
    KEY,nombre VARCHAR(100) NOT NULL,  
    id_departamento INT,  
    FOREIGN KEY (id_departamento) REFERENCES  
    departamentos(id_departamento)  
);
```

En este caso:

- **FOREIGN KEY (id_departamento) REFERENCES departamentos(id_departamento)** significa que la columna **id_departamento** en la tabla **empleados** hace referencia a la columna **id_departamento** en la tabla **departamentos**. Esto asegura que solo se pueden insertar registros en **empleados** si el **id_departamento** existe en la tabla **departamentos**.

Ejemplo 4: Crear una Tabla con Restricciones de Unicidad

En algunas ocasiones, es posible que queramos asegurarnos de que una columna no contenga valores duplicados. Para esto, podemos usar la restricción **UNIQUE**. Esta restricción garantiza que todos los valores de una columna sean únicos.

Instrucciones:

- Imaginemos que queremos garantizar que los correos electrónicos de los empleados sean únicos en la tabla **empleados**.

SQL para Crear la Tabla con Restricción de Unicidad:

```
CREATE TABLE empleados (  
    id_empledo INT NOT NULL PRIMARY  
    KEY,nombre VARCHAR(100) NOT NULL,  
    correo_electronico VARCHAR(100) UNIQUE  
);
```

En este caso:

- **correo_electronico VARCHAR(100) UNIQUE** asegura que no se podrán insertar dos empleados con el mismo correo electrónico en la tabla.
-

Actualización de datos (UPDATE)

Cuando trabajemos con bases de datos, en muchos casos necesitaremos **modificar los datos** que ya están almacenados. Para esto, utilizamos la sentencia **UPDATE** en SQL. Esta sentencia nos permite actualizar uno o más registros de una tabla, cambiando los valores de las columnas según las condiciones que definamos.

Sintaxis Básica de UPDATE

La sintaxis de la sentencia **UPDATE** es bastante sencilla. Necesitaremos especificar:

1. El nombre de la tabla que contiene los datos que queremos actualizar.
2. Las columnas que vamos a modificar y sus nuevos valores.
3. La condición (usando **WHERE**) que especifica qué registros deben ser actualizados.

Sintaxis General:

```
UPDATE nombre_de_tabla  
SET columna1 = nuevo_valor1, columna2 =  
nuevo_valor2, ... WHERE condición;
```

- **nombre_de_tabla**: El nombre de la tabla que contiene los registros que queremos actualizar.
- **columnaX**: El nombre de la columna que vamos a modificar.
- **nuevo_valorX**: El nuevo valor que se asignará a la columna.
- **WHERE**: Condición que filtra los registros que serán actualizados. Si omitimos esta cláusula, **todos** los registros de la tabla serán modificados.

Ejemplo 1: Actualizar un Solo Registro

Supongamos que tenemos una tabla llamada **empleados** con las siguientes columnas: **id_empleado**, **nombre**, y **salario**. Si queremos actualizar el salario de un empleado específico, usamos el **UPDATE** con una condición **WHERE** que identifique al empleado.

Instrucciones:

- Queremos actualizar el salario del empleado con **id_empleado = 1**.
- Supongamos que el nuevo salario del empleado es **3000**.

SQL para Actualizar el Registro:

UPDATE

empleados

SET

salario = 3000

WHERE id_empleado =

1; En este caso:

- **UPDATE empleados** indica que vamos a modificar la tabla **empleados**.
- **SET salario = 3000** significa que la columna **salario** se actualizará con el valor **3000**.
- **WHERE id_empleado = 1** asegura que solo se actualizará el registro cuyo **id_empleado** sea igual a 1.

Ejemplo 2: Actualizar Varios Registros

Supongamos que queremos aumentar el salario de todos los empleados que trabajan en el departamento de **Ventas** en un 10%. En este caso, actualizamos varios registros a la vez utilizando una condición en la cláusula **WHERE**.

Instrucciones:

- Queremos incrementar el salario en un 10% de todos los empleados que estén en el departamento **Ventas**.

SQL para Actualizar Múltiples Registros:

UPDATE empleados

SET salario = salario * 1.10

WHERE departamento =

'Ventas';

En este caso:

- **SET salario = salario * 1.10** indica que el salario de todos los empleados del departamento de **Ventas** se incrementará en un 10%.
- **WHERE departamento = 'Ventas'** asegura que solo los empleados cuyo **departamento** sea **Ventas** sean

actualizados. Ejemplo 3: Actualizar Varios

Campos en un Registro

Supongamos que tenemos una tabla llamada **empleados** con las columnas **id_empleado**, **nombre**, **departamento**, y **salario**. Si queremos actualizar más de una columna al mismo tiempo, podemos hacerlo en una sola sentencia **UPDATE**.

Instrucciones:

- Queremos actualizar tanto el nombre como el salario de un empleado con **id_empleado = 3**.
- Supongamos que el nuevo nombre es **Ana López** y el nuevo salario es **4000**.

SQL para Actualizar Múltiples Columnas:

UPDATE empleados

SET nombre = 'Ana López', salario =
4000 WHERE id_empleado = 3;

En este caso:

- **SET nombre = 'Ana López', salario = 4000** indica que el nombre se actualizará a **Ana López** y el salario se actualizará a **4000**.
- **WHERE id_empleado = 3** asegura que solo se actualizará el empleado con **id_empleado = 3**.

Ejemplo 4: Actualizar Todos los Registros (Sin WHERE)

Si omitimos la cláusula **WHERE**, **todos** los registros de la tabla serán actualizados. Esto puede ser útil en ciertas situaciones, pero también puede llevar a errores si no se usa con cuidado.

Instrucciones:

- Supongamos que, por alguna razón, necesitamos actualizar el salario de **todos los empleados** de la tabla **empleados** a **5000**.

SQL para Actualizar Todos los Registros:

```
UPDATE  
empleados SET  
salario = 5000;
```

En este caso:

- **SET salario = 5000** establece el salario de todos los empleados a **5000**.
- No hay cláusula **WHERE**, por lo que **todos los registros** de la tabla **empleados** serán modificados.

Ejemplo 5: Usar Subconsultas para Actualizar Datos

En algunos casos, puede que queramos actualizar los datos basándonos en valores de otra tabla. Esto se puede lograr utilizando subconsultas dentro de la sentencia **UPDATE**.

Instrucciones:

- Supongamos que tenemos una tabla llamada **empleados** y otra llamada **aumentos**, y queremos actualizar el salario de los empleados en la tabla

empleados basado en los aumentos registrados en la tabla **aumentos**.

- Queremos que el salario de cada empleado se actualice según el aumento correspondiente al **id_empleado** en la tabla **aumentos**.

SQL para Actualizar Usando una Subconsulta:

```
UPDATE empleados
```

```
SET salario = salario
```

```
+ (
```

```
  SELECT
```

```
  aumento FROM
```

```
  aumentos
```

```
  WHERE aumentos.id_empleado = empleados.id_empleado
```

```
)
```

```
WHERE id_empleado IN (SELECT id_empleado FROM
```

```
aumentos);
```

En este caso:

- **SET salario = salario + (...)** aumenta el salario del empleado según el valor que se obtiene de la subconsulta.
- La subconsulta (**SELECT aumento FROM aumentos WHERE aumentos.id_empleado = empleados.id_empleado**) obtiene el aumento correspondiente de la tabla **aumentos** para cada empleado.
- **WHERE id_empleado IN (...)** asegura que solo los empleados que están presentes en la tabla **aumentos** sean actualizados.

Consideraciones Importantes al Usar UPDATE

1. **Siempre usar WHERE con precaución:** Si no agregamos la cláusula **WHERE**, **todos** los registros de la tabla serán actualizados. Es importante verificar siempre que la condición **WHERE** sea correcta antes de ejecutar la consulta.
2. **Transacciones:** Si estamos actualizando muchos registros y queremos tener la opción de deshacer los cambios en caso de error, es recomendable usar **transacciones** (**BEGIN TRANSACTION**, **COMMIT**, **ROLLBACK**).

3. **Pruebas antes de actualizar:** Es recomendable **hacer una consulta SELECT primero** con las mismas condiciones que usaremos en la sentencia **UPDATE** para asegurarnos de que estamos seleccionando los registros correctos.
4. **Respaldo de datos:** Antes de realizar actualizaciones importantes en la base de datos, especialmente en entornos de producción, siempre es recomendable tener un respaldo de los datos.

Borrado de datos (DROP vs. DELETE vs. TRUNCATE)

Cuando trabajemos con bases de datos, es posible que en algún momento necesitemos **eliminar datos** de una tabla o incluso **eliminar la tabla completa**. Para esto, en SQL tenemos tres opciones principales: **DROP**, **DELETE** y **TRUNCATE**. Cada una de estas operaciones tiene un comportamiento diferente, y es importante entender sus diferencias para utilizarlas correctamente según la situación.

1. DROP: Eliminar una Tabla o un Objeto de la Base de Datos

La sentencia **DROP** se utiliza para eliminar completamente una tabla, vista, índice o cualquier otro objeto de la base de datos. Cuando utilizamos **DROP**, no solo eliminamos los **datos** de la tabla, sino que **también eliminamos la propia tabla** y cualquier relación o estructura asociada a ella, como índices, claves foráneas, etc.

Características de DROP:

- Elimina la **tabla completa** (o cualquier objeto de la base de datos).
- Elimina **todos los datos** de la tabla de manera permanente.
- **No se puede revertir:** Si eliminamos una tabla con **DROP**, los datos se pierden de forma definitiva (a menos que tengamos un respaldo).

Sintaxis:

DROP TABLE

nombre_de_tabla; Ejemplo de

uso:

Supongamos que tenemos una tabla llamada **empleados** y necesitamos eliminarla completamente de la base de datos.

DROP TABLE empleados;

En este caso, la tabla **empleados** se elimina junto con **todos sus datos y estructuras asociadas** (índices, claves, etc.).

2. DELETE: Eliminar Datos de una Tabla con Condiciones

La sentencia **DELETE** se utiliza para eliminar **filas** específicas de una tabla, pero **sin eliminar la tabla** en sí. Esta sentencia es útil cuando queremos eliminar **solo algunos registros** de la tabla según una condición específica.

Características de DELETE:

- Elimina **filas** individuales de una tabla.
- No elimina la **estructura** de la tabla.
- **Es posible utilizar una condición WHERE** para eliminar solo registros específicos. Si no se especifica una condición **WHERE**, se eliminarán **todos** los registros de la tabla, pero la tabla seguirá existiendo.
- Las eliminaciones **son registradas en el log de transacciones**, por lo que **pueden ser revertidas** si estamos trabajando dentro de una transacción.

Sintaxis:

DELETE FROM nombre_de_tabla WHERE
condición; Ejemplo de uso:

Supongamos que tenemos la tabla **empleados** y necesitamos eliminar a todos los empleados que tengan un salario inferior a 2000.

DELETE FROM empleados

WHERE salario < 2000;

En este caso, solo se eliminarán los registros que cumplan la condición **salario < 2000**. Los demás registros permanecerán en la tabla **empleados**.

Importante: Si ejecutamos el siguiente comando **sin WHERE**, todos los registros serán eliminados, pero la tabla seguirá existiendo:

DELETE FROM empleados;

3. TRUNCATE: Eliminar Todos los Datos de una Tabla Rápidamente

La sentencia **TRUNCATE** también se utiliza para eliminar todos los registros de una tabla, pero de una manera **más rápida y eficiente** que **DELETE**. A diferencia de **DELETE**, **TRUNCATE** no elimina registros de forma fila por fila, sino que **elimina todos los registros** de la tabla en un solo paso.

Características de TRUNCATE:

- Elimina **todos los registros** de la tabla **de forma rápida**.
- **No se puede usar con condiciones:** Es una operación que afecta a toda la tabla.
- **No genera registros en el log de transacciones** como lo hace **DELETE**, por lo que es más rápido, pero también **no puede revertirse** de la misma manera.
- **Mantiene la estructura de la tabla:** La tabla y sus definiciones (como las columnas, índices, claves) permanecen intactas. Sin embargo, los contadores de identidad (como en campos AUTO_INCREMENT) suelen **restablecerse** a su valor inicial.
- **No dispara triggers:** A diferencia de **DELETE**, que puede activar triggers, **TRUNCATE** no lo hace.

Sintaxis:

TRUNCATE TABLE

nombre_de_tabla;Ejemplo de uso:

Si necesitamos eliminar todos los registros de la tabla **empleados**, podemos usar **TRUNCATE** de la siguiente forma:

TRUNCATE TABLE empleados;

En este caso, **todos los registros** de la tabla **empleados** se eliminarán rápidamente, pero la estructura de la tabla (como columnas y claves) permanecerá intacta.

Comparación entre DROP, DELETE y TRUNCATE

Característica	DROP	DELETE	TRUNCATE
¿Qué elimina?	Elimina la tabla completa.	Elimina filas específicas o todas, pero no la tabla.	Elimina todos los registros de la tabla.
¿Se puede revertir?	No (permanente).	Sí, si se usa dentro de una transacción.	No (permanente).
¿Afecta la estructura de la tabla?	Sí, elimina la tabla.	No, la tabla sigue existiendo.	No, la tabla sigue existiendo.
¿Es más rápido?	No aplica, ya que elimina la tabla.	Depende de la cantidad de registros.	Sí, es más rápido que DELETE.
¿Restablece los contadores de identidad?	No aplica.	No.	Sí, generalmente.
¿Activa triggers?	No.	Sí.	No.
¿Se puede usar con condiciones?	No.	Sí, con la cláusula WHERE.	No.

¿Cuándo usar cada uno?

- **Usar DROP:** Cuando necesitamos **eliminar la tabla por completo**, junto con su estructura y todos los datos. Es útil cuando ya no necesitamos más la tabla en la base de datos.
- **Usar DELETE:** Cuando queremos eliminar **algunos registros específicos** de una tabla o **todos** los registros, pero **sin eliminar la estructura de la tabla**. Ideal cuando hay que hacer un borrado selectivo o cuando se necesita la opción de revertir la operación dentro de una transacción.
- **Usar TRUNCATE:** Cuando necesitamos **eliminar todos los registros** de una tabla de manera **rápida y eficiente**, y no necesitamos la posibilidad de revertir la operación. Es útil cuando la tabla tiene muchos registros y queremos hacer una limpieza completa.

Modificación de tablas (ALTER TABLE)

Cuando trabajemos con bases de datos, es probable que necesitemos **modificar la estructura de una tabla** después de haberla creado. Para hacer esto, usamos la sentencia **ALTER TABLE** en SQL. Esta sentencia nos permite realizar cambios en una tabla ya existente, como agregar, eliminar o modificar columnas, cambiar el nombre de la tabla, entre otros.

¿Qué Podemos Hacer con ALTER TABLE?

El comando **ALTER TABLE** tiene varias opciones que nos permiten modificar la estructura de una tabla sin perder los datos existentes. Las operaciones más comunes incluyen:

1. **Agregar columnas** a una tabla.
2. **Eliminar columnas** de una tabla.
3. **Modificar el tipo de datos de una columna.**
4. **Renombrar columnas.**
5. **Renombrar la tabla.**
6. **Añadir restricciones** (como claves primarias, foráneas, o de unicidad).

Sintaxis Básica de ALTER TABLE

La sintaxis general para usar **ALTER TABLE** depende de lo que queramos hacer, pero sigue un patrón básico como este:

```
ALTER TABLE  
nombre_de_tabla  
[opción_a_realizar];
```

Donde:

- **nombre_de_tabla**: Es el nombre de la tabla que queremos modificar.
- **opción_a_realizar**: Es la operación específica que queremos hacer, como agregar o eliminar columnas, o modificar una columna existente.

A continuación, vamos a ver ejemplos prácticos de cada una de las operaciones más comunes con **ALTER TABLE**.

1. Agregar una Columna a una Tabla

Si necesitamos agregar una nueva columna a una tabla existente, usamos la opción **ADD**. Es una operación bastante común cuando queremos almacenar más información en una tabla sin perder los datos actuales.

Sintaxis:

```
ALTER TABLE nombre_de_tabla  
ADD nombre_columna  
tipo_de_dato;
```

Ejemplo de uso:

Supongamos que tenemos una tabla **empleados** y queremos agregar una nueva columna llamada **fecha_ingreso** de tipo DATE para almacenar la fecha en que cada empleado ingresó a la empresa.


```
ALTER TABLE empleados  
ADD fecha_ingreso DATE;
```

Esto agrega la columna **fecha_ingreso** a la tabla **empleados**, pero no afectará los datos existentes. Los registros antiguos tendrán valores NULL en esta nueva columna hasta que se les asigne un valor.

2. Eliminar una Columna de una Tabla

Si decidimos que ya no necesitamos una columna en una tabla, podemos usar la opción

DROP COLUMN para eliminarla.

Sintaxis:

```
ALTER TABLE nombre_de_tabla  
DROP COLUMN  
nombre_columna;
```

Ejemplo de uso:

Supongamos que la tabla **empleados** tiene una columna **edad** que ya no es necesaria. Para eliminarla, usamos la siguiente instrucción:

```
ALTER TABLE  
empleados  
DROP  
COLUMN edad;
```

Después de ejecutar este comando, la columna **edad** y todos los datos almacenados en ella serán eliminados de la tabla **empleados**.

3. Modificar el Tipo de Datos de una Columna

En ocasiones, podemos necesitar cambiar el tipo de datos de una columna, por

ejemplo, si la columna originalmente fue definida como VARCHAR(50) pero necesitamos más espacio, o si queremos cambiar un tipo INTEGER por FLOAT.

Sintaxis:

```
ALTER TABLE nombre_de_tabla  
MODIFY COLUMN nombre_columna nuevo_tipo_de_dato;
```

Ejemplo de uso:

Imaginemos que tenemos una columna **salario** en la tabla **empleados**, que fue definida originalmente como INT (entero), pero ahora necesitamos almacenarlo como DECIMAL para incluir decimales. El comando sería:

```
ALTER TABLE empleados  
MODIFY COLUMN salario DECIMAL(10, 2);
```

Este comando cambiará el tipo de la columna **salario** para que pueda almacenar valores decimales con 2 dígitos después del punto.

4. Renombrar una Columna

Si decidimos cambiar el nombre de una columna, podemos hacerlo con la opción **RENAME COLUMN** (dependiendo del sistema de gestión de bases de datos, puede variar).

Sintaxis:

```
ALTER TABLE nombre_de_tabla  
RENAME COLUMN nombre_columna_actual TO nuevo_nombre_columna;
```

Ejemplo de uso:

Supongamos que la columna **nombre_empleado** en la tabla **empleados** se llama ahora

nombre_completo, para que el nombre sea más claro. El comando sería:

```
ALTER TABLE empleados  
RENAME COLUMN nombre_empleado TO nombre_completo;
```

Este comando cambiará el nombre de la columna **nombre_empleado** a **nombre_completo**.

5. Renombrar la Tabla

Si necesitamos cambiar el nombre de una tabla, podemos hacerlo con la opción **RENAMETO**.

Sintaxis:

```
ALTER TABLE  
nombre_tabla_actual RENAME TO  
nuevo_nombre_tabla;
```

Ejemplo de uso:

Supongamos que la tabla **empleados** ha cambiado de propósito y ahora se usa para almacenar información de **trabajadores**, por lo que necesitamos renombrarla. El comando sería:

```
ALTER TABLE  
empleados RENAME TO  
trabajadores;
```

Este comando cambiará el nombre de la tabla **empleados** a **trabajadores**.

6. Agregar Restricciones (Constraints)

Podemos usar **ALTER TABLE** para agregar restricciones, como claves primarias, claves foráneas, restricciones de unicidad, etc. Las restricciones ayudan a asegurar la integridad de los datos en la base de datos.

Sintaxis para agregar una clave primaria:

```
ALTER TABLE nombre_de_tabla  
ADD CONSTRAINT nombre_constraint PRIMARY KEY (columna);
```

Ejemplo de uso:

Supongamos que tenemos una tabla **empleados** y queremos agregar una **clave primaria** en la columna **id_empleado**.

```
ALTER TABLE empleados  
ADD CONSTRAINT pk_empleados PRIMARY KEY (id_empleado);
```

Este comando agrega una clave primaria en la columna **id_empleado** de la tabla **empleados**, asegurando que los valores en esta columna sean únicos.

Sintaxis para agregar una clave foránea:

```
ALTER TABLE nombre_de_tabla  
ADD CONSTRAINT nombre_constraint FOREIGN KEY (columna)  
REFERENCES otra_tabla(otra_columna);
```

Ejemplo de uso:

Supongamos que tenemos una tabla **empleados** y otra tabla **departamentos**, y queremos agregar una clave foránea en la columna **id_departamento** de la tabla **empleados** que haga referencia a la columna **id_departamento** de la tabla **departamentos**.

ALTER TABLE empleados

ADD CONSTRAINT fk_departamento FOREIGN KEY

(id_departamento)REFERENCES

departamentos(id_departamento);

Este comando agrega una clave foránea en la columna **id_departamento** de la tabla **empleados**, estableciendo una relación con la columna **id_departamento** de la tabla **departamentos**.

Resumen de Operaciones con ALTER TABLE

Operación	Descripción	Sintaxis
Agregar columna	Añade una nueva columna ala tabla	ALTER TABLE tabla ADD columna tipo_dato;
Eliminar columna	Elimina una columna existente de la tabla	ALTER TABLE tabla DROP COLUMN columna;
Modificar tipo de columna	Cambia el tipo de datos de una columna	ALTER TABLE tabla MODIFY COLUMN columna nuevo_tipo;
Renombrar columna	Cambia el nombre de una columna	ALTER TABLE tabla RENAME COLUMN columna TO nuevo_nombre;
Renombrar tabla	Cambia el nombre de la tabla	ALTER TABLE tabla RENAME TO nuevo_nombre;
Agregar restricción (PK/FK)	Agrega restricciones como clave primaria o foránea	ALTER TABLE tabla ADD CONSTRAINT restricción tipo(columna);

Operaciones con más de una tabla

Cuando trabajamos con bases de datos relacionales, es muy común que los datos estén distribuidos en varias tablas, y en muchas ocasiones necesitamos realizar operaciones que involucren más de una tabla. SQL nos ofrece varias formas de trabajar con múltiples tablas, permitiéndonos combinar, filtrar y relacionar datos de distintas tablas.

Operaciones Comunes con Varias Tablas en SQL

Las operaciones más comunes con más de una tabla en SQL incluyen:

1. **JOIN** (Unir tablas)
2. **UNION** (Combinar resultados de consultas)
3. **Subconsultas** (Consultas dentro de otras consultas)

1. Uso de JOIN para Combinar Tablas

El comando **JOIN** se utiliza para combinar filas de dos o más tablas basadas en una condición común entre ellas. Este es uno de los conceptos más importantes cuando trabajamos con bases de datos relacionales. Los JOIN se utilizan para extraer datos relacionados entre tablas.

Tipos de JOIN:

- **INNER JOIN**: Devuelve solo las filas que tienen una coincidencia en ambas tablas.
- **LEFT JOIN** (o **LEFT OUTER JOIN**): Devuelve todas las filas de la primera tabla y las filas coincidentes de la segunda tabla. Si no hay coincidencia, los resultados de la segunda tabla serán NULL.
- **RIGHT JOIN** (o **RIGHT OUTER JOIN**): Devuelve todas las filas de la segunda tabla y las filas coincidentes de la primera tabla.
- **FULL JOIN** (o **FULL OUTER JOIN**): Devuelve todas las filas cuando hay una coincidencia en cualquiera de las tablas. Si no hay coincidencia, devuelve NULL en el lado que no tiene coincidencias.

Sintaxis General de JOIN:

SELECT

columnas FROM

tabla1 JOIN

tabla2

ON tabla1.columna =

tabla2.columna;Ejemplo con

INNER JOIN:

Supongamos que tenemos dos tablas: **empleados** y **departamentos**.

- La tabla **empleados** tiene las siguientes columnas: id_employado, nombre,id_departamento.
- La tabla **departamentos** tiene las siguientes columnas: id_departamento,nombre_departamento.

Queremos obtener una lista de todos los empleados junto con el nombre de su departamento.

```
SELECT empleados.nombre,  
departamentos.nombre_departamentoFROM empleados  
INNER JOIN departamentos  
ON empleados.id_departamento = departamentos.id_departamento;
```

Este comando une las tablas **empleados** y **departamentos** a través de la columna **id_departamento**, devolviendo solo los registros donde existe una coincidencia en ambas tablas.

Ejemplo con LEFT JOIN:

Supongamos que queremos listar todos los empleados, incluso aquellos que no están asignados a ningún departamento. Utilizamos **LEFT JOIN** para obtener todos los empleados y, si no tienen un departamento asignado, mostrar NULL en la columna del departamento.

```
SELECT empleados.nombre,  
departamentos.nombre_departamentoFROM empleados  
LEFT JOIN departamentos  
ON empleados.id_departamento = departamentos.id_departamento;
```

Con **LEFT JOIN**, se muestran todos los empleados, y si un empleado no tiene un departamento asignado, el valor en **nombre_departamento** será NULL.

2. Uso de UNION para Combinar Resultados de Consultas

La operación **UNION** se utiliza para combinar los resultados de dos o más consultas SQL. A diferencia de un **JOIN**, **UNION** no se basa en una condición de coincidencia entre las tablas. En lugar de eso, simplemente toma los resultados de varias consultas y los concatena. Es importante que las consultas involucradas tengan el **mismo número de columnas** y tipos de datos compatibles.

Sintaxis de UNION:

```
SELECT  
columnasFROM  
tabla1 UNION  
SELECT  
columnasFROM  
tabla2;
```

Ejemplo de uso de UNION:

Supongamos que tenemos dos tablas: **empleados_2023** y **empleados_2024**, que almacenan los empleados de los años 2023 y 2024, respectivamente. Queremos obtener una lista de todos los empleados de ambos años.

```
SELECT nombre  
FROM  
empleados_2023  
UNION  
SELECT nombre  
FROM empleados_2024;
```

El comando **UNION** combina los resultados de ambas consultas y devuelve

una lista de nombres de empleados sin duplicados. Si quisiéramos incluir los duplicados, usaríamos **UNION ALL**:

```
SELECT nombre
FROM
empleados_2023
UNION ALL
SELECT nombre
FROM empleados_2024;
```

Con **UNION ALL**, obtendremos todos los registros, incluidos los duplicados.

3. Subconsultas (Subqueries) para Consultas Anidadas

Las **subconsultas** son consultas dentro de otras consultas. Son útiles cuando necesitamos realizar una consulta basada en los resultados de otra. Las subconsultas se pueden usar en la cláusula **WHERE**, **FROM**, o **SELECT**.

Subconsulta en la cláusula WHERE:

Supongamos que tenemos la tabla **empleados** y queremos encontrar todos los empleados cuyo salario sea mayor que el salario promedio de todos los empleados.

```
SELECT nombre,
salarioFROM
empleados
WHERE salario > (SELECT AVG(salario) FROM empleados);
```

En este ejemplo, la subconsulta (**SELECT AVG(salario) FROM empleados**) calcula el salario promedio, y la consulta principal selecciona todos los empleados cuyo salario es mayor que el promedio.

Subconsulta en la cláusula FROM:

A veces es necesario usar una subconsulta en la cláusula **FROM** para tratar los resultados de una subconsulta como una tabla temporal.

Supongamos que tenemos dos tablas: **empleados** y **departamentos**, y queremos obtener el salario promedio por departamento. Primero, calculamos el salario promedio en cada departamento utilizando una subconsulta en **FROM**:

```
SELECT dept.id_departamento, dept.nombre_departamento,  
AVG(emp.salario) AS salario_promedio  
FROM departamentos dept  
JOIN (SELECT id_departamento, salario FROM  
empleados) emp ON dept.id_departamento =  
emp.id_departamento  
GROUP BY dept.id_departamento;
```

En este caso, la subconsulta (**SELECT id_departamento, salario FROM empleados**) se usa como una tabla temporal que luego se une con la tabla **departamentos**.

4. Operaciones con INNER JOIN, LEFT JOIN y RIGHT JOIN

Ejemplo completo con INNER JOIN, LEFT JOIN y RIGHT JOIN:

Imaginemos que tenemos tres tablas en una base de datos: **clientes**, **pedidos** y **productos**.

- **clientes**: id_cliente, nombre_cliente
- **pedidos**: id_pedido, id_cliente, fecha_pedido
- **productos**: id_producto, nombre_producto

Queremos obtener una lista de clientes que han realizado pedidos, junto con los productos que han pedido y la fecha de su pedido. Usamos un **INNER JOIN** para unir los clientes con los pedidos, y luego un **LEFT JOIN** para incluir también los productos, incluso si algún cliente no ha pedido un producto específico.

```
SELECT clientes.nombre_cliente, pedidos.fecha_pedido,  
productos.nombre_productoFROM clientes  
INNER JOIN pedidos ON clientes.id_cliente = pedidos.id_cliente  
LEFT JOIN productos ON pedidos.id_pedido =  
productos.id_producto;
```

1. **INNER JOIN**: Une a los clientes con los pedidos, pero solo devuelve los clientesque tienen pedidos.
2. **LEFT JOIN**: Incluye todos los clientes con pedidos, aunque no todos hayan pedidoun producto específico.

Tipos de Uniones en SQL

Cuando trabajamos con bases de datos que contienen múltiples tablas, las **uniones** son esenciales para combinar y obtener información relacionada de esas tablas. SQL ofrece varios tipos de **uniones** y técnicas para combinar datos de diferentes tablas, cada una conun propósito específico.

1. INNER JOIN

El **INNER JOIN** se utiliza para devolver solo las filas que tienen coincidencias en ambastablas que estamos uniendo. Si no hay coincidencia, esos registros no aparecerán en el resultado final.

Sintaxis:

```
SELECT  
columnas FROM  
tabla1 INNER  
JOIN tabla2  
ON tabla1.columna =  
tabla2.columna;Ejemplo:
```

Imaginemos dos tablas: **empleados** y **departamentos**.

- **empleados** tiene las columnas: id_empleado, nombre_empleado, id_departamento.

- **departamentos** tiene las columnas: id_departamento, nombre_departamento.

Queremos obtener los nombres de los empleados junto con el nombre de su departamento, pero solo para los empleados que están asignados a un departamento.

```
SELECT empleados.nombre_empleado,  
departamentos.nombre_departamento  
FROM empleados  
INNER JOIN departamentos  
ON empleados.id_departamento = departamentos.id_departamento;
```

El **INNER JOIN** solo devolverá los empleados que tengan un id_departamento que coincida con el de la tabla **departamentos**.

2. OUTER JOINS

Los **OUTER JOINS** se utilizan cuando queremos obtener todas las filas de una tabla, junto con las filas coincidentes de otra tabla, **incluso si no hay coincidencia**. Existen tres tipos de OUTER JOINS:

a) LEFT JOIN (o LEFT OUTER JOIN)

Devuelve todas las filas de la tabla de la izquierda, y las filas coincidentes de la tabla de la derecha. Si no hay coincidencia, devuelve NULL en las columnas de la tabla de la derecha.

Sintaxis:

```
SELECT  
columnas  
FROM  
tabla1 LEFT  
JOIN tabla2  
ON tabla1.columna =  
tabla2.columna; Ejemplo:
```

Queremos obtener una lista de todos los empleados, incluso si no están asignados a un departamento. Para esto usamos **LEFT JOIN**.

```
SELECT empleados.nombre_empleado,  
departamentos.nombre_departamento FROM empleados  
LEFT JOIN departamentos  
ON empleados.id_departamento = departamentos.id_departamento;
```

Este comando devolverá todos los empleados, y si un empleado no tiene un departamento asignado, el campo **nombre_departamento** será NULL.

b) RIGHT JOIN (o RIGHT OUTER JOIN)

Devuelve todas las filas de la tabla de la derecha, y las filas coincidentes de la tabla de la izquierda. Si no hay coincidencia, devuelve NULL en las columnas de la tabla de la izquierda.

Sintaxis:

```
SELECT  
columnas FROM  
tabla1 RIGHT  
JOIN tabla2  
ON tabla1.columna = tabla2.columna;
```

Ejemplo:

Queremos obtener todos los departamentos, incluyendo aquellos que no tienen empleados asignados. Para esto usamos **RIGHT JOIN**.

```
SELECT empleados.nombre_empleado,  
departamentos.nombre_departamento FROM empleados  
RIGHT JOIN departamentos  
ON empleados.id_departamento = departamentos.id_departamento;
```

Este comando devolverá todos los departamentos, y si un departamento no tiene empleados asignados, el campo **nombre_empleado** será NULL.

c) FULL JOIN (o FULL OUTER JOIN)

Devuelve todas las filas cuando hay una coincidencia en una de las tablas. Si no hay coincidencia, devuelve NULL en las columnas de la tabla que no tiene coincidencia.

Sintaxis:

```
SELECT  
columnas FROM  
tabla1 FULL  
JOIN tabla2  
ON tabla1.columna =  
tabla2.columna; Ejemplo:
```

Queremos obtener una lista de todos los empleados y todos los departamentos, **incluso sino tienen coincidencias**. Usamos **FULL JOIN**:

```
SELECT empleados.nombre_empleado,  
departamentos.nombre_departamento FROM empleados  
FULL JOIN departamentos  
ON empleados.id_departamento = departamentos.id_departamento;
```

Este comando devolverá todos los empleados y todos los departamentos, con NULL donde no haya coincidencias en la tabla contraria.

3. CROSS JOIN

El **CROSS JOIN** devuelve el producto cartesiano de dos tablas, es decir, combina cada fila de la primera tabla con cada fila de la segunda tabla. Esto puede generar un número muy grande de filas si las tablas tienen muchos registros.

Sintaxis:

```
SELECT columnas
```

```
FROM tabla1
```

```
CROSS JOIN
```

```
tabla2;
```

Ejemplo:

Si tenemos una tabla **colores** con los colores Rojo, Verde, Azul, y una tabla **tamaños** con los tamaños Pequeño, Mediano, Grande, un **CROSS JOIN** devolverá todas las combinaciones posibles entre colores y tamaños.

```
SELECT colores.nombre_color,  
tamaños.nombre_tamañoFROM colores  
CROSS JOIN tamaños;
```

Este comando devolverá todas las combinaciones de colores y tamaños:

- Rojo - Pequeño
- Rojo - Mediano
- Rojo - Grande
- Verde - Pequeño
- ... y así sucesivamente.

4. UNION vs UNION ALL

Ambos **UNION** y **UNION ALL** se utilizan para combinar los resultados de dos o más consultas, pero hay una diferencia clave:

- **UNION**: Combina los resultados de varias consultas y **elimina los duplicados**.
- **UNION ALL**: Combina los resultados de varias consultas **sin eliminar duplicados**.

Sintaxis de UNION:

```
SELECT  
columnasFROM  
tabla1 UNION  
SELECT  
columnasFROM  
tabla2;
```

Sintaxis de UNION ALL:

```
SELECT  
columnasFROM  
tabla1 UNION  
ALL  
SELECT  
columnas FROM  
tabla2; Ejemplo de  
UNION:
```

Si tenemos dos tablas **empleados_2023** y **empleados_2024**, y queremos obtener una listade todos los empleados sin duplicados, usamos **UNION**:

```
SELECT nombre  
FROM  
empleados_2023  
UNION  
SELECT nombre  
FROM empleados_2024;
```

Esto combinará los resultados de ambas tablas, **eliminando los duplicados**.Ejemplo de UNION ALL:

Si queremos obtener todos los empleados de ambas tablas, **incluyendo los duplicados**,usamos **UNION ALL**:

```
SELECT nombre  
FROM
```



```
empleados_2023
UNION ALL
SELECT nombre
FROM empleados_2024;
```

Este comando incluirá todos los registros de ambas tablas, incluso si un empleado aparece en ambas tablas.

5. Expresiones en SQL

SQL nos permite usar expresiones como **CASE** y **COALESCE** para realizar operaciones condicionales y manejar valores nulos en las consultas.

a) CASE

La expresión **CASE** se usa para realizar operaciones condicionales dentro de una consulta SQL. Es como un "if-else" en programación.

Sintaxis:

```
SELECT
    nombre,
    salario,
    CASE
        WHEN salario > 5000 THEN 'Alto'
        WHEN salario BETWEEN 3000 AND 5000 THEN
            'Medio' ELSE 'Bajo'
    END AS
categoria_salario FROM
empleados; Explicación:
```

En este ejemplo, clasificamos a los empleados según su salario. Si el salario es mayor a 5000, se clasifica como "Alto". Si está entre 3000 y 5000, como "Medio", y si no, como "Bajo".

b) COALESCE

La función **COALESCE** devuelve el primer valor no nulo en una lista de expresiones. Es útil cuando queremos reemplazar valores nulos por un valor predeterminado.

Sintaxis:

```
SELECT nombre,  
       COALESCE(telefono, 'No disponible') AS  
telefono FROM empleados;
```

Explicación:

En este caso, si la columna **telefono** es NULL, el resultado será '**No disponible**' en lugar de NULL.

6. Funciones Predefinidas en Bases de Datos

Las bases de datos SQL incluyen una serie de funciones predefinidas que se pueden usar para realizar operaciones comunes, como manipulación de cadenas, cálculos, manejo de fechas y más.

Ejemplos comunes de funciones predefinidas:

- **COUNT()**: Cuenta el número de filas.

```
SELECT COUNT(*) FROM empleados;
```

- **SUM()**: Suma los valores de una columna numérica.

```
SELECT SUM(salario) FROM empleados;
```

- **AVG()**: Calcula el promedio de una columna numérica.

```
SELECT AVG(s
```

Agrupaciones y Particionamiento de Bases de Datos en SQL

Cuando trabajamos con bases de datos en SQL, a menudo necesitamos realizar operaciones para **agrupar** y **particionar** los datos de manera que podamos obtener resultados significativos y eficientes.

Agrupando Datos (GROUP BY)

Cuando necesitemos agrupar los datos en función de una o más columnas, **GROUP BY** será nuestra herramienta principal. Este comando nos permite agrupar filas que tienen los mismos valores en las columnas especificadas y realizar cálculos de agregación, como **COUNT()**, **SUM()**, **AVG()**, entre otros.

Sintaxis:

```
SELECT columna1, columna2,  
función_agregada(columna) FROM tabla  
GROUP BY columna1,  
columna2; Ejemplo:
```

Imaginemos que tenemos una tabla llamada **ventas** que tiene las siguientes columnas: **id_venta**, **producto**, **cantidad**, **precio**, **fecha**.

Si queremos obtener el total de ventas por cada producto, haremos una agrupación por la columna **producto**.

```
SELECT producto, SUM(cantidad * precio) AS  
total_ventas FROM ventas  
GROUP BY producto;
```

Este comando nos devolverá el total de ventas por cada producto, agrupando los resultados por el nombre del **producto**.

Filtrando Datos Agrupados (HAVING)

A veces, después de realizar una agrupación, necesitamos **filtrar los resultados** de las agrupaciones basadas en una condición. Para hacer esto, usaremos la cláusula **HAVING**. La diferencia clave entre **WHERE** y **HAVING** es que **WHERE** filtra antes de la agrupación, mientras que **HAVING** filtra después de la agrupación.

Sintaxis:

```
SELECT columna1, columna2,  
función_agregada(columna) FROM tabla  
GROUP BY columna1  
HAVING función_agregada(columna)  
condición; Ejemplo:
```

Siguiendo el ejemplo anterior, supongamos que solo queremos mostrar los productos cuya venta total supera los \$5000.

```
SELECT producto, SUM(cantidad * precio) AS  
total_ventas FROM ventas  
GROUP BY producto  
HAVING SUM(cantidad * precio) > 5000;
```

Este comando devolverá solo los productos cuya venta total es mayor a \$5000.

Cláusula OVER

La cláusula **OVER** se utiliza en combinación con **funciones de ventana** para realizar cálculos en un conjunto de filas que está relacionado con la fila actual sin tener que agruparlos resultados. Las funciones de ventana permiten realizar cálculos acumulados o comparativos dentro de un conjunto de registros.

Sintaxis:

```
SELECT columna, función_ventana() OVER (PARTITION BY columna  
ORDER BY columna)  
FROM
```

tabla;

Ejemplo:

Supongamos que queremos calcular el **promedio acumulado** de ventas de cada producto, sin agrupar los resultados, sino mostrando los valores de cada fila junto con el promedio acumulado.

```
SELECT producto, cantidad,  
       AVG(cantidad) OVER (PARTITION BY producto ORDER BY fecha)  
AS promedio_acumulado  
FROM ventas;
```

Este comando calculará el promedio acumulado de la cantidad de ventas por producto, ordenado por la fecha de cada venta.

Funciones de Ventana (FIRST_VALUE, LAST_VALUE, LAG, LEAD)

Las **funciones de ventana** nos permiten realizar cálculos avanzados que nos proporcionan información sobre las filas de un conjunto de datos relativo a la fila actual.

- **FIRST_VALUE()**: Devuelve el primer valor de un conjunto de filas.
- **LAST_VALUE()**: Devuelve el último valor de un conjunto de filas.
- **LAG()**: Devuelve el valor de una fila anterior en el conjunto.
- **LEAD()**: Devuelve el valor de una fila siguiente en el conjunto.

Ejemplos:

FIRST_VALUE():

Queremos ver el primer valor de ventas de cada producto. SELECT producto, cantidad,

```
       FIRST_VALUE(cantidad) OVER (PARTITION BY producto ORDER BY fecha)  
AS primer_venta  
FROM ventas;
```

LAST_VALUE():

Queremos ver el último valor de ventas de cada producto. SELECT producto, cantidad,

LAST_VALUE(cantidad) OVER (PARTITION BY producto ORDER BY fecha ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING) AS

ultima_venta

FROM

ventas;

LAG():

Queremos ver la venta del día anterior para cada producto. SELECT producto, fecha, cantidad,

LAG(cantidad, 1) OVER (PARTITION BY producto ORDER BY fecha) AS
venta_anteri

o r F R O M

v e n t a s ;

LEAD():

Queremos ver la venta del día siguiente para cada producto. SELECT producto, fecha, cantidad,

LEAD(cantidad, 1) OVER (PARTITION BY producto ORDER BY fecha) AS
venta_siguie

teFROM

ventas;

Ordenar Registros Agrupados (RANK, ROW_NUMBER)

Cuando trabajamos con datos agrupados, podemos usar funciones como

RANK() o **ROW_NUMBER()** para asignar un número de fila a cada registro, lo que puede ser útil para ordenar o clasificar los resultados.

- **RANK()**: Asigna un ranking a cada fila en función de un orden específico. Si hay filas con el mismo valor, obtienen el mismo rango y se salta el siguiente número.
- **ROW_NUMBER()**: Asigna un número único a cada fila, sin importar si hay filas con el mismo valor.

Ejemplos:

ROW_NUMBER():

Queremos asignar un número de fila a cada venta de producto, ordenado por la cantidad de ventas de mayor a menor.

```
SELECT producto, cantidad,  
       ROW_NUMBER() OVER (PARTITION BY producto ORDER BY cantidad  
DESC)AS numero_fila  
FROM  
ventas;  
RANK():
```

Queremos asignar un ranking a los productos basado en la cantidad total de ventas, donde los productos con la misma cantidad de ventas tienen el mismo ranking.

```
SELECT producto, SUM(cantidad * precio) AS total_ventas,  
       RANK() OVER (ORDER BY SUM(cantidad * precio) DESC) AS  
rankingFROM ventas  
GROUP BY producto;
```

Vistas (VIEWS)

Las **vistas** son consultas almacenadas que pueden ser utilizadas como si fueran tablas. Nos permiten simplificar y reutilizar consultas complejas, y pueden ser muy útiles cuando necesitamos mostrar un conjunto de resultados sin modificar las tablas originales.

Sintaxis:

```
CREATE VIEW nombre_vista  
ASSELECT columnas  
FROM tabla  
WHERE  
condición;
```

Ejemplo:

Supongamos que tenemos una vista que muestra todos los productos vendidos con su total de ventas.

```
CREATE VIEW vista_ventas AS
SELECT producto, SUM(cantidad * precio) AS
total_ventas FROM ventas
GROUP BY producto;
```

Ahora, podemos consultar la vista como si fuera una tabla:

```
SELECT * FROM vista_ventas;
```

Funciones de Usuario (FUNCTIONS)

Las **funciones de usuario** nos permiten crear funciones personalizadas para realizar operaciones complejas que no están disponibles de forma predeterminada en SQL.

Sintaxis:

```
CREATE FUNCTION nombre_función
(parametros) RETURNS tipo_dato
AS
BEGIN
    -- Lógica de la
    función RETURN
    valor;
END;
```

Ejemplo:

Supongamos que queremos una función que calcule el salario total de un empleado, considerando un bono y una deducción:


```

CREATE FUNCTION calcular_salario_total(salario_base DECIMAL, bono
DECIMAL,deducción DECIMAL)
RETURNS
DECIMALAS
BEGIN
    RETURN salario_base + bono -
deducción;END;

```

Ahora podemos usar esta función en nuestras consultas:

```

SELECT nombre_empleado, calcular_salario_total(salario, bono,
deducción) ASsalario_total
FROM empleados;

```

Procedimientos Almacenados (STORED PROCEDURES)

Los **procedimientos almacenados** son conjuntos de instrucciones SQL que se guardan en la base de datos y pueden ser ejecutadas cuando sea necesario. Nos permiten encapsular lógica compleja y reutilizarla.

Sintaxis:

```

CREATE PROCEDURE nombre_procedimiento
(parametros)AS
BEGIN
    -- Lógica del
procedimientoEND;

```

Ejemplo:

Supongamos que queremos crear un procedimiento que actualice el salario de un empleado:

```

CREATE PROCEDURE actualizar_salario(id_empleado INT, nuevo_salario
DECIMAL)AS
BEGIN
    UPDATE empleados

```

```
SET salario = nuevo_salario  
WHERE id_empleado =  
id_empleado;END;
```

Podemos ejecutar este procedimiento de la siguiente manera:

```
EXEC actualizar_salario 1, 5000;
```

Disparadores (Triggers)

Los **disparadores** (triggers) son procedimientos almacenados que se ejecutan automáticamente en respuesta a ciertos eventos, como la inserción, actualización o eliminación de datos en una tabla.

Sintaxis:

```
CREATE TRIGGER  
nombre_triggerAFTER INSERT  
ON tabla  
FOR EACH  
ROWBEGIN  
    -- Lógica del  
triggerEND;
```

Ejemplo:

Queremos crear un disparador que registre el historial de cambios en los salarios de los empleados:

```
CREATE TRIGGER  
historial_salarioAFTER UPDATE  
ON empleados FOR EACH ROW  
BEGIN  
    INSERT INTO historial_salarios (id_empleado, salario_anterior,  
    salario_nuevo)VALUES (OLD.id_empleado, OLD.salario, NEW.salario);
```

END;

Este disparador se ejecutará automáticamente después de cada actualización en la tabla

empleados, guardando el historial de cambios de salario.

Administración de Bases de Datos en SQL

La administración de bases de datos es una parte fundamental para asegurar que los datos se gestionen de manera eficiente, segura y optimizada. Como administradores de bases de datos, necesitaremos realizar diversas tareas como crear bases de datos, realizar respaldos, restaurar datos, importar y exportar archivos, y gestionar permisos de acceso.

1. Cómo Crear una Base de Datos

Crear una base de datos es uno de los primeros pasos en la administración de una base de datos SQL. Usaremos el comando **CREATE DATABASE** para crear una nueva base de datos en el servidor.

Sintaxis:

```
CREATE DATABASE
```

nombre_base_de_datos;Ejemplo:

Imaginemos que queremos crear una base de datos para gestionar la información de una tienda. Usaremos el siguiente comando:

```
CREATE DATABASE tienda;
```

Este comando creará una nueva base de datos llamada **tienda**. Después de crear la base de datos, podremos crear tablas y otros objetos dentro de ella.

2. Respalidar una Base de Datos

Es fundamental hacer **respaldos** regulares de las bases de datos para evitar la pérdida de información en caso de fallos del sistema. Usaremos el comando **BACKUP DATABASE** para crear una copia de seguridad de la base de datos.

Sintaxis:

```
BACKUP DATABASE  
nombre_base_de_datos TO DISK =  
'ruta_del_archivo.bak';
```

Ejemplo:

Si queremos respaldar la base de datos **tienda** en un archivo llamado **tienda_backup.bak**, lo haremos con el siguiente comando:

```
BACKUP DATABASE tienda  
TO DISK = 'C:\backups\tienda_backup.bak';
```

Este comando creará un archivo de respaldo de la base de datos **tienda** en la ruta especificada. Es una buena práctica realizar respaldos completos periódicamente.

3. Restaurar una Base de Datos

En caso de que se pierdan datos o se presente un problema en la base de datos, necesitaríamos **restaurar** la base de datos desde un archivo de respaldo.

Usaremos el comando **RESTORE DATABASE** para restaurar una base de datos a partir de un archivo de respaldo.

Sintaxis:

```
RESTORE DATABASE  
nombre_base_de_datos FROM DISK =  
'ruta_del_archivo.bak';
```

Ejemplo:

Si necesitamos restaurar la base de datos **tienda** desde un archivo de respaldo llamado

tienda_backup.bak, el comando sería:

```
RESTORE DATABASE tienda  
FROM DISK = 'C:\backups\tienda_backup.bak';
```

Este comando restaurará la base de datos **tienda** al estado que tenía en el momento en que se creó el respaldo. Es importante verificar que el archivo de respaldo esté en la ubicación correcta antes de ejecutar la restauración.

4. Importar y Exportar Archivos

A menudo necesitaremos **importar** y **exportar** datos desde o hacia archivos, como archivos CSV, Excel o archivos de texto. Usaremos comandos como **BULK INSERT** y **bcp** (Bulk Copy Program) para realizar estas tareas.

a) Importar Datos (BULK INSERT)

Para importar datos desde un archivo externo, utilizamos **BULK INSERT**. Este comando permite cargar grandes cantidades de datos de manera eficiente.

Sintaxis:

```
BULK INSERT  
nombre_tabla FROM  
'ruta_del_archivo'  
WITH (FIELDTERMINATOR = 'terminador_de_campo',  
ROWTERMINATOR = 'terminador_de_fila');
```

Ejemplo:

Si tenemos un archivo CSV llamado **clientes.csv** y queremos cargarlo en una tabla **clientes**, el comando sería:

```
BULK INSERT clientes  
FROM 'C:
```

```
\datos\clientes.csv'
```

```
WITH (FIELDTERMINATOR = ',', ROWTERMINATOR = '\n');
```

Este comando importará los datos del archivo **clientes.csv** a la tabla **clientes**, considerando que los campos están separados por comas y las filas por saltos de línea.

b) Exportar Datos (bcp)

Si necesitamos exportar datos desde una base de datos a un archivo, podemos usar la utilidad **bcp** (Bulk Copy Program). **bcp** se usa a menudo desde la línea de comandos para exportar e importar datos de manera eficiente.

Sintaxis:

Desde la línea de comandos de SQL Server:

```
bcp nombre_base_de_datos.dbo.nombre_tabla out 'ruta_del_archivo' -S servidor -U  
usuario
```

```
-P contraseña
```

```
-cEjemplo:
```

Para exportar los datos de la tabla **clientes** a un archivo CSV:

```
bcp tienda.dbo.clientes out 'C:\datos\clientes_exportados.csv' -S mi_servidor -U  
mi_usuario
```

```
-P mi_contraseña -c
```

Este comando exportará los datos de la tabla **clientes** a un archivo CSV en la ruta especificada.

5. Administración de Permisos

La **gestión de permisos** es una parte crítica de la administración de bases de datos. Asegurarnos de que los usuarios tengan acceso adecuado a los datos es

fundamental para la seguridad. Usaremos comandos como **GRANT**, **REVOKE** y **DENY** para administrar permisos de acceso.

a) Conceder Permisos (GRANT)

El comando **GRANT** nos permite otorgar permisos a los usuarios sobre objetos específicos, como tablas o bases de datos.

Sintaxis:

GRANT permiso ON objeto TO
usuario; Ejemplo:

Supongamos que queremos conceder permisos de lectura a un usuario llamado **usuario1** sobre la tabla **clientes**. Usaremos el siguiente comando:

```
GRANT SELECT ON clientes TO usuario1;
```

Este comando le dará al usuario **usuario1** permisos de **SELECT** sobre la tabla **clientes**, lo que significa que podrá consultar los datos de esa tabla.

b) Revocar Permisos (REVOKE)

El comando **REVOKE** se usa para **eliminar** los permisos previamente concedidos a un usuario.

Sintaxis:

REVOKE permiso ON objeto FROM usuario;

Ejemplo:

Si queremos revocar los permisos de **SELECT** otorgados anteriormente al usuario **usuario1**, usaremos el siguiente comando:

REVOKE SELECT ON clientes FROM usuario1;

Este comando eliminará los permisos de **SELECT** sobre la tabla **clientes** para el usuario **usuario1**.

c) Denegar Permisos (DENY)

El comando **DENY** se utiliza para negar explícitamente ciertos permisos, incluso si el usuario tiene otros permisos en el objeto.

Sintaxis:

DENY permiso ON objeto TO

usuario; Ejemplo:

Si queremos denegar el permiso de **DELETE** a **usuario1** sobre la tabla **clientes**, usaremos el siguiente comando:

DENY DELETE ON clientes TO usuario1;

Este comando evitará que el usuario **usuario1** pueda eliminar registros de la tabla **clientes**, incluso si tiene permisos adicionales.

Modelado de una Base de Datos Relacional en SQL

Cuando trabajamos con bases de datos relacionales, uno de los pasos clave es **modelar adecuadamente** la estructura de los datos. Esto implica entender cómo organizar los datos, establecer relaciones entre ellos y asegurarnos de que se cumplan ciertas reglas para garantizar la integridad de la información. A continuación, exploraremos los aspectos fundamentales del **modelado de bases de datos relacionales**:

- **Modelo Entidad-Relación (ER)**
- **Llaves (Primarias y Secundarias)**

- **Constraints (DEFAULT, NOT NULL, CHECK, UNIQUE)**
- **Índices**
- **Normalización**

Modelo Entidad-Relación (ER)

El **Modelo Entidad-Relación (ER)** es una forma de representar de manera visual y estructurada los elementos que componen nuestra base de datos y las relaciones entre ellos. En este modelo:

- Las **entidades** representan objetos o conceptos importantes en nuestro sistema (porejemplo, **Cientes, Productos**).
- Las **relaciones** indican cómo se conectan estas entidades entre sí (por ejemplo, **Compra, Vende**).

Ejemplo:

Imaginemos que estamos modelando una base de datos para una tienda en línea. Usaremos las siguientes entidades y relaciones:

- **Entidad: Cientes** (con atributos como id_cliente, nombre, email).
- **Entidad: Productos** (con atributos como id_producto, nombre, precio).
- **Relación: Realiza** (que conecta a Cientes con Productos).

Esto puede representarse de manera gráfica con un diagrama ER, donde cada entidad es un rectángulo, y las relaciones son líneas que conectan esas entidades.

Llaves (Primarias y Secundarias)

Las **llaves** son esenciales para identificar de manera única los registros dentro de una tabla. Hay dos tipos principales de llaves en bases de datos relacionales:

Llave Primaria (Primary Key)

Una **llave primaria** es un campo (o conjunto de campos) que se utiliza para identificar de forma única cada registro en una tabla. No puede haber dos registros con el mismo valor en una llave primaria.

Ejemplo:

Imaginemos una tabla de **Cientes**, donde cada cliente tiene un identificador único **id_cliente**:

```
CREATE TABLE Cientes
( id_cliente INT PRIMARY
  KEY,nombre VARCHAR(100),
  email VARCHAR(100)
);
```

En este caso, **id_cliente** es la llave primaria, ya que garantiza que cada cliente sea único en la tabla.

Llave Secundaria (Foreign Key)

Una **llave secundaria** es un campo que establece una relación entre dos tablas. Se utiliza para vincular una fila de una tabla con otra fila en una tabla diferente.

Ejemplo:

Supongamos que tenemos una tabla **Pedidos** que se relaciona con la tabla **Cientes** a través de la columna **id_cliente**. Usaremos una llave secundaria para vincular un pedido a un cliente específico:

```
CREATE TABLE Pedidos
( id_pedido INT PRIMARY
  KEY,fecha DATE,
  id_cliente INT,
  FOREIGN KEY (id_cliente) REFERENCES Cientes(id_cliente)
);
```

En este caso, **id_cliente** es una llave secundaria que establece una relación entre la tabla

Pedidos y la tabla

Cientes. Constraints

(Restricciones)

Las **constraints** o restricciones son reglas que se aplican a los datos para asegurar su integridad y validez. A continuación, exploraremos algunas de las restricciones más comunes:

DEFAULT

La **restricción DEFAULT** se usa para asignar un valor predeterminado a una columna cuando no se proporciona un valor explícito durante la inserción de un registro.

Ejemplo:

Si queremos que el campo **estado** de un **pedido** tenga un valor predeterminado de **'pendiente'** cuando no se especifique un estado, lo definimos así:

```
CREATE TABLE Pedidos
( id_pedido INT PRIMARY
  KEY, fecha DATE,
  estado VARCHAR(50) DEFAULT 'pendiente'
);
```

Ahora, si insertamos un pedido sin especificar el **estado**, automáticamente tomará el valor

'pendiente'.

NOT NULL

La restricción **NOT NULL** asegura que una columna no pueda tener valores nulos. Es útil cuando necesitamos que ciertos campos siempre tengan datos.

Ejemplo:

Supongamos que queremos asegurarnos de que el **nombre** de un cliente no pueda ser nulo:CREATE TABLE Clientes (

```
id_cliente INT PRIMARY KEY,  
nombre VARCHAR(100) NOT  
NULL,email VARCHAR(100)  
);
```

Con esto, si intentamos insertar un cliente sin nombre, obtendremos un error.CHECK

La restricción **CHECK** se usa para asegurarse de que los valores de una columna cumplan con una condición específica.

Ejemplo:

Si queremos asegurarnos de que el **precio** de un producto sea siempre mayor que 0,podemos usar la restricción **CHECK** de la siguiente manera:

```
CREATE TABLE Productos  
( id_producto INT PRIMARY  
KEY,nombre VARCHAR(100),  
precio DECIMAL(10,  
2),CHECK (precio > 0)  
);
```

Esto evitará que se inserten productos con precios negativos o nulos.UNIQUE

La restricción **UNIQUE** garantiza que los valores en una columna sean únicos en toda la tabla, es decir, no puede haber dos registros con el mismo valor en esa columna.

Ejemplo:

Si queremos que el campo **email** de la tabla **Clientes** sea único, podemos definirlo así:CREATE TABLE Clientes (

```
id_cliente INT PRIMARY  
KEY,nombre VARCHAR(100),  
email VARCHAR(100) UNIQUE  
);
```

Esto evitará que se inserten dos clientes con el mismo

email. Índices

Los **índices** son estructuras de datos que mejoran la velocidad de las consultas de búsqueda en las tablas. Ayudan a acelerar las operaciones de lectura, pero pueden afectar el rendimiento en las operaciones de escritura (como **INSERT**, **UPDATE**, **DELETE**).

Sintaxis para crear un índice:

```
CREATE INDEX nombre_indice ON tabla  
(columna);
```

Ejemplo:

Si queremos mejorar el rendimiento de las búsquedas por **email** en la tabla **Cientes**, podemos crear un índice en esa columna:

```
CREATE INDEX idx_email ON Cientes (email);
```

Ahora, las consultas que busquen clientes por **email** serán más rápidas.

Normalización

La **normalización** es el proceso de organizar los datos en una base de datos para minimizar la redundancia y las dependencias. La normalización se logra a través de las **formas normales** (1NF, 2NF, 3NF), que definen un conjunto de reglas para estructurar los datos.

1ra Forma Normal (1NF)

Una tabla está en **1ra. Forma Normal (1NF)** si todos los atributos de la tabla contienen valores atómicos (es decir, indivisibles) y si cada columna tiene un único valor por registro.

Ejemplo de 1NF:

Supongamos que tenemos una tabla de **Pedidos** que almacena varios productos por pedido en una sola celda (lo cual es incorrecto):

id_pedido	productos
1	"Producto A, Producto B"
2	"Producto C"

Para cumplir con la **1NF**, debemos dividir los productos en registros separados:

id_pedido	producto
1	Producto A
1	Producto B
2	Producto C

2da Forma Normal (2NF)

Una tabla está en **2da. Forma Normal (2NF)** si está en **1NF** y no tiene dependencias parciales. Esto significa que todos los atributos no clave deben depender completamente de la llave primaria.

Ejemplo de 2NF:

Imaginemos una tabla de **Pedidos** que almacena tanto el **id_cliente** como el **nombre_cliente**:

id_pedido	id_cliente	nombre_cliente	producto
1	101	Juan Pérez	Producto A
1	101	Juan Pérez	Producto B

Para cumplir con la **2NF**, debemos separar el **nombre_cliente** en una tabla diferente para evitar la redundancia:

```
CREATE TABLE Clientes
( id_cliente INT PRIMARY
KEY, nombre_cliente
VARCHAR(100)
);
```

```
CREATE TABLE Pedidos
( id_pedido INT PRIMARY
KEY, id_cliente INT,
producto VARCHAR(100),
FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
);
```

3ra Forma Normal (3NF)

Una tabla está en **3ra. Forma Normal (3NF)** si está en **2NF** y no tiene dependencias transitivas. Es decir, los atributos no clave deben depender directamente de la llave primaria y no de otros atributos no clave.

Ejemplo de 3NF:

Si tenemos una tabla de **Pedidos** que también almacena la **ciudad_cliente** (que depende de **id_cliente**):

id_pedido	id_cliente	ciudad_cliente	producto
1	101	Madrid	Producto A

Para cumplir con la **3NF**, debemos crear una tabla separada para almacenar la **ciudad_cliente** y evitar dependencias transitivas:

```
CREATE TABLE Ciudades
( id_cliente INT PRIMARY
KEY, ciudad_cliente
```

```
    VARCHAR(100)  
);
```

```
CREATE TABLE Pedidos  
  ( id_pedido INT PRIMARY  
    KEY,id_cliente INT,  
    producto VARCHAR(100),  
    FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)  
  );
```